# 分布式系统概念

## 划重点

1. 一章：分布式系统概念 通信类型、原语
2. 二章：分布式系统的模型 分布式系统建模：形式化描述，描述计算，进程，进程之间通信和状态 进程和通信，关键：分布式执行过程的状态：因果序，因果性的定义，消息传递的类型 FFO CO Non-FFO 分布式全局状态（**重要，给分布式时空图会找全局一致状态**）全局一致状态切面。因果序，全局一致，**切面**
3. 三章：时钟对齐，**lamport（逻辑时钟）和向量时钟要求会计算**，矩阵时钟不要求，**虚拟时钟要求掌握，重点看**：相关概念和技术；SK的实现；Local control 机制；虚拟实现的机制：**antimessage和rollback很重要** 物理时钟对齐
4. 四章：快照协议 CL算法的原理，SK算法原理，实现全局一致快照，**能够画出来全局一致的快照集合，给时空图，用切线或zigpath找**
5. 五章：几个概念，考察少
6. 六章：同步类型和异步类型算法的特征，不用细节和计算，**给例子知道原理和过程**
7. 七章：消息序及其怎么保证，**时空图判断CO或CO伪逆，RSC（Realized Synchronized Communication）是什么概念：异步系统怎么转同步系统，找crown，给一个例子找crown判断RSC(充要条件)**，CO保障和组通信算法；最优的KS算法不考

## 其他知识点

- **逻辑并发**：当两个事件间无因果影响时，两个事件是逻辑并发的。
- **物理并发**：不同事件在物理时间的同一时刻发生。

## 第一章：分布式系统概念

分布式系统概念 通信类型、原语

- **分布式系统**：
  - **概念**：使用互联网络将分布在不同位置的计算机连接起来
  - **特点**：物理分布，逻辑集中；个体独立，整体统一
  - **关键特征**：无公共物理时钟，无共享内存（进程间通过消息传递通信），地理分离，自主性和异构性
  - **通信类型**：异步执行（没有同步时钟，消息延迟有限但无界，进程执行异步的时间没有上界）/同步执行（时钟漂移有界，消息投递发生在同一个逻辑步，进程执行一步的时间有已知上界）
  - **通信原语**：

    <u>同步 / 异步</u>：（同步给予通信逻辑上的正确性）

    同步执行意味着收发双方必须见面，Send只有在Receive完成后才算完成；

    异步执行意味着当数据从用户缓冲区拷贝到内核缓冲区后，控制权就返回给进程，发送方无需等待接收方

    <u>阻塞 / 非阻塞</u>：（非阻塞给予处理器并行计算的能力）

    阻塞意味着调用原语后，进程会被**挂起**，直到操作（无论是Sync握手还是Async拷贝）完成，控制权才返回。

非阻塞意味着调用原语后，控制权**立即返回**，不管操作是否完成，通常返回一个 handle（句柄），后续通过 Wait(handle) 或轮询来检查操作是否真正完成。

四种组合：同步阻塞最严格，同步非阻塞，异步阻塞最常用，异步非阻塞最快

# 第二章：全局一致状态

---

分布式建模的形式化描述/因果序/消息传递的类型/全局一致性状态的判断

## 一、 分布式系统的形式化描述

**1. 三种事件**

- **内部事件:** 仅改变进程自身状态。
- **发送事件:** 改变发送方状态 + 改变信道状态。
- **接收事件 :** 改变接收方状态 + 改变信道状态。

**2. 状态**

- **进程状态 ($LS_i$):** 也就是内存、寄存器的值。
- **信道状态 ($SC_{ij}$):** 正在传输中的消息集合（即：发了但还没收到的消息）。

---

## 二、 因果序 —— 核心考点

这是Lamport最著名的定义，用于解决“谁在前，谁在后”的问题。

**1. 因果关系的定义
记作 $e_i \to e_j$ (Event $i$ happens before $j$)，满足以下任一条件：

1. **本地顺序:** 同一个进程内，先发生的事件 $\to$ 后发生的事件。
2. **消息传递:** 消息 $m$ 的 $send(m) \to rec(m)$。
3. **传递性:** 如果 $a \to b$ 且 $b \to c$，则 $a \to c$。

**2. 并发事件**

- **定义:** 如果事件 $a$ 和 $b$ 互不影响，即 $a \nrightarrow b$ 且 $b \nrightarrow a$，则称它们是**并发的**，记作 $a||b$。
- **判别技巧:** 在时空图上，如果两个点之间**没有**任何路径（无论是通过进程线还是消息箭头）能连通，它们就是并发的。

---

## 三、 消息传递模型—— 重要考点

1. **Non-FIFO (非先进先出):** 每个信道都运行一个集合，其中发送进程向集合加入消息，接收进程从中移除消息， 被加入或移除的消息顺序是随机的； 。
2. **FIFO (先进先出):** 每个信道是个“队列”。$P_1$ 发给 $P_2$ 的消息 $m_1, m_2$，如果 $send(m_1)$ 在先，那么 $rec(m_1)$ 必在先。
3. **CO (Causal Ordering, 因果序) —— 最强模型:**
   - **定义:** 如果发送事件 $send(m_1) \to send(m_2)$（注意：这不仅仅是同一个进程发的，可能是 $P_1$发了$m_1$告诉$P_2$，然后$P_2$发了$m_2$给$P_3$），那么目的地必须先收到 $m_1$ 再收到 $m_2$。
   - **包含关系: CO $\subset$ FIFO $\subset$ Non-FIFO。**

- **重点:** 满足CO一定满足FIFO，但满足FIFO不一定满足CO。

---

# 四、 全局状态与一致性切面 —— 考试重灾区

这是你提到的**最重要的考点**：给时空图，找全局一致状态/切面。

## 1. 什么是“切面” (Cut)?

- 在时空图上画一条**锯齿状的线 (Zigzag line)**，横跨所有进程线。
- 切面将时间分为 **PAST (过去)** 和 **FUTURE (未来)**。

## 2. 全局一致状态 (Consistent Global State) 的判别法则

**法则 : “不能有果无因”。**

- **一致 (Consistent):** 如果在切面的 PAST 部分记录了某个消息的**接收 ($rec$)**，那么该消息的**发送 ($send$)** 也必须在 PAST 部分。
- **不一致 (Inconsistent):** 如果切面切断了一条消息箭头，且箭头的**箭头（接收）在 PAST**，而**箭尾（发送）在 FUTURE**。
  - *通俗解释：* 这意味着系统记录了“我收到了信”，但还没有记录“信被寄出”。这在物理上是不可能的。

## 3. 做题技巧：如何在图上找/画 Consistent Cut?

1. **画线法：** 用笔在几个进程间画一条线（这就是切面）。
2. **检查箭头：** 观察所有穿过这条切面的**消息箭头**。
   - **情况 A (左发右收):** 箭头从左（Past）穿到右（Future）。即：发送了，还没收到。**这是允许的**，这代表消息在信道中 (Channel State)。
   - **情况 B (右发左收):** 箭头从右（Future）穿到左（Past）。即：还没发送，就已经收到了。**这是绝对禁止的。**
3. **判断：** 只要出现**一个**情况 B 的箭头，这个切面就是**不一致**的。如果没有情况 B，就是**一致**的。

**示例分析:**

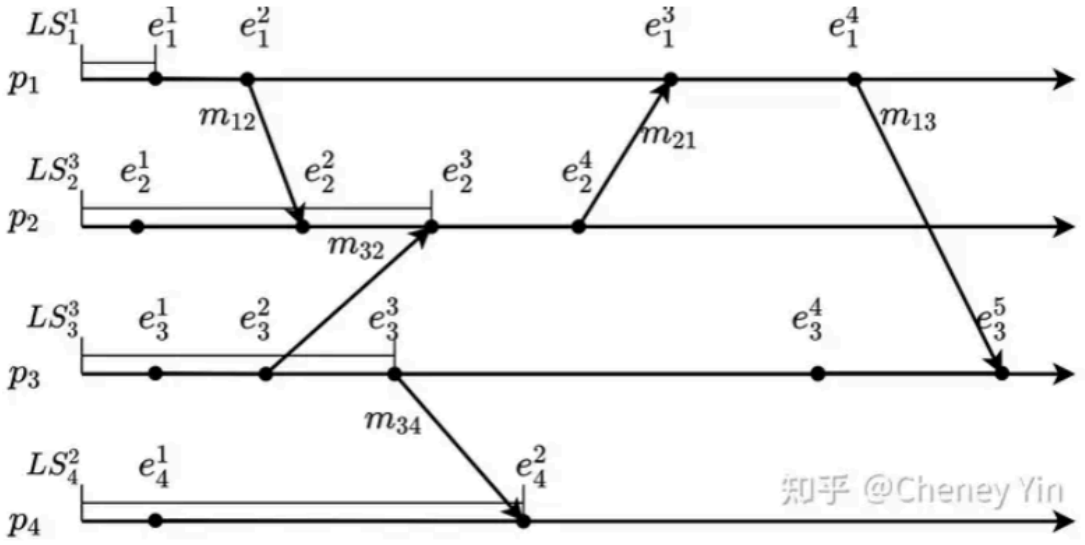



图2 某分布式运行的时空图

在图2中，包含本地状态$\{LS_1^1, LS_2^3, LS_3^3, LS_4^2\}$的全局状态$GS_1$是不一致的，因为$p_2$的状态$LS_2^3$已经记录了消息$m_{12}$的接收，然而$p_1$的状态$LS_1^1$还未记录该消息的发送，所有不满足一致性条件，即没有发送消息，就没有接收消息。

在图2中，包含本地状态$\{LS_1^2, LS_2^4, LS_3^4, LS_4^2\}$的全局状态$GS_2$是一致状态，因为除了包含消息$m_{21}$的$C_{21}$外的其余信道都是空的（空信道意味着发送的消息都已被接收）。

---

## 五、 补充重点：Past/Future Cones (锥)

课件花了大量篇幅讲这个，这也是潜在考点。

- **Past Cone (过去锥):** 对于事件 $e_j$，所有能**影响**到它的事件集合（也就是所有满足 $\rightarrow e_j$ 的点）。
- **Future Cone (未来锥):** 对于事件 $e_j$，所有受它**影响**的事件集合（也就是所有满足 $e_j \rightarrow$ 的点）。
- **Concurrent (并发区域):** 除去 Past Cone 和 Future Cone 之外的所有空白区域。
- **Max_Past / Min_Future:** 这两个概念描述了锥的“表面”或“边界”。

- **全序集**：集合中所有事件均可以比较发生的顺序（因果和并发）： ei<ej 或 ej<ei ，具有传递性：ei<ej ， ej<ek =>ei<ek ； 用于全局一致性判断；反对称性：一个事件不可同时满足： ei<ej 且 ej<ei
- **偏序集**：集合中的事件只能部分比较因果性，并发事件不能比较因果性；用于因果一致性判断；因果性具有传递性，并发没有

| 特性 | 偏序（Partial Order） | 全序（Total Order） |
| --- | --- | --- |
| **顺序性** | 并发事件无法比较 | 任意两个事件都可以比较其先后顺序 |
| **场景** | 因果关系跟踪（如矢量时钟） | 全局一致性（如分布式事务、日志） |
| **性能** | 高效，避免不必要的顺序化处理 | 开销较高，需要额外的同步机制 |
| **使用场景** | 因果一致性模型 | 强一致性模型 |

- **偏序 -> 全序转化**：添加额外时钟

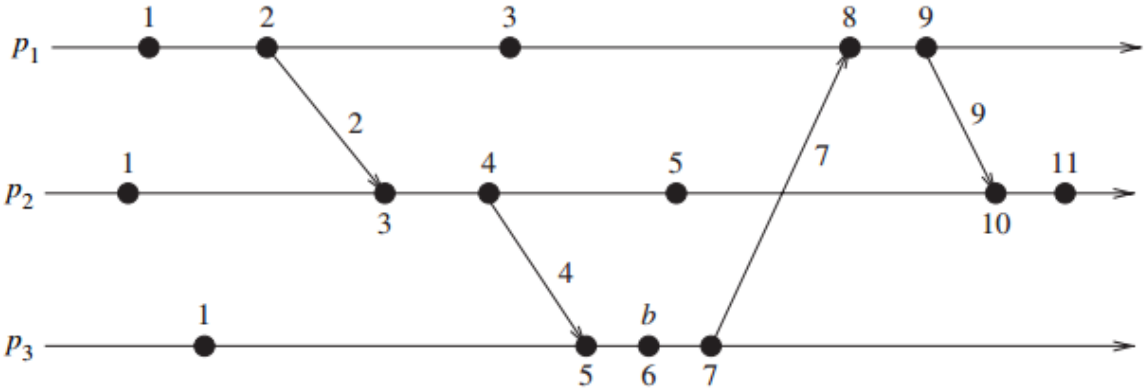
- **通信模型**：
  - **同步通信**：消息发送结束后，发送进程阻塞等待接收进程接受到消息才继续下一步发送；简单，但性能不好
  - **异步通信**：消息发送后，发送进程可以继续发送消息，消息会在适当时候被接收进程接收；高并行度； buffer 管理问题，因为发送进程可能突然发送大量消息

# 第三章：逻辑时钟

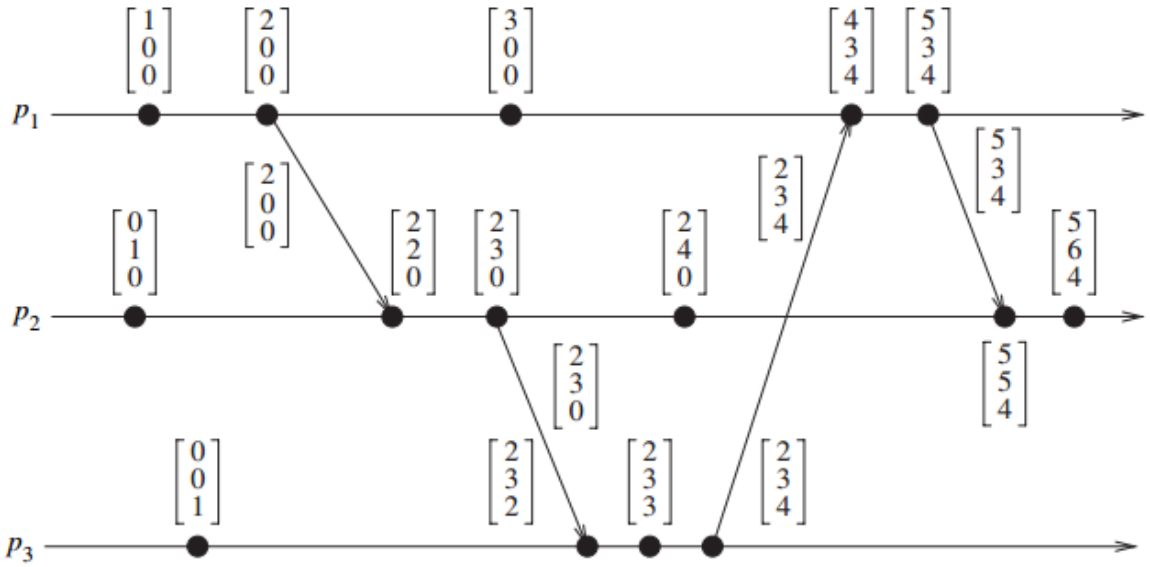
- 分布式系统不存在全局共享的内存和物理时钟，使用逻辑时钟判断分布式系统的因果单调性
- 标量时钟和向量时钟都用于描述分布式系统中事件发生的因果顺序，信道中传输时间戳

## 第一部分：逻辑时钟计算（必考：Lamport 与 向量时钟）

- **标量时钟(Slide 8-9)**
- **R1 (本地事件):** 进程每执行一次事件（包括发送、内部事件），计数器加 1。
  - $C_i := C_i + 1$
- **R2 (接收消息):** 收到消息时，消息里会带着发送方的时钟 $C_{msg}$。接收方比较自己的时钟和消息的时钟，取最大值，然后加 1。
  - $C_i := \max(C_i, C_{msg}) + 1$




- 无法判断并发
- **向量时钟 (Slide 16)**
- **R1 (本地事件/发送消息前):**
  - 只增加自己的位置：$VT_i[i] := VT_i[i] + 1$
- **R2 (接收消息):**
  - 收到消息带来了一个向量 $VT_{msg}$。
  - **第一步 (对齐):** 每一个位置 $k$，都取最大值。$VT_i[k] = \max(VT_i[k], VT_{msg}[k])$。
  - **第二步 (自增):** 增加自己的位置（表示接收这个动作本身也是一个事件）。$VT_i[i] := VT_i[i] + 1$。

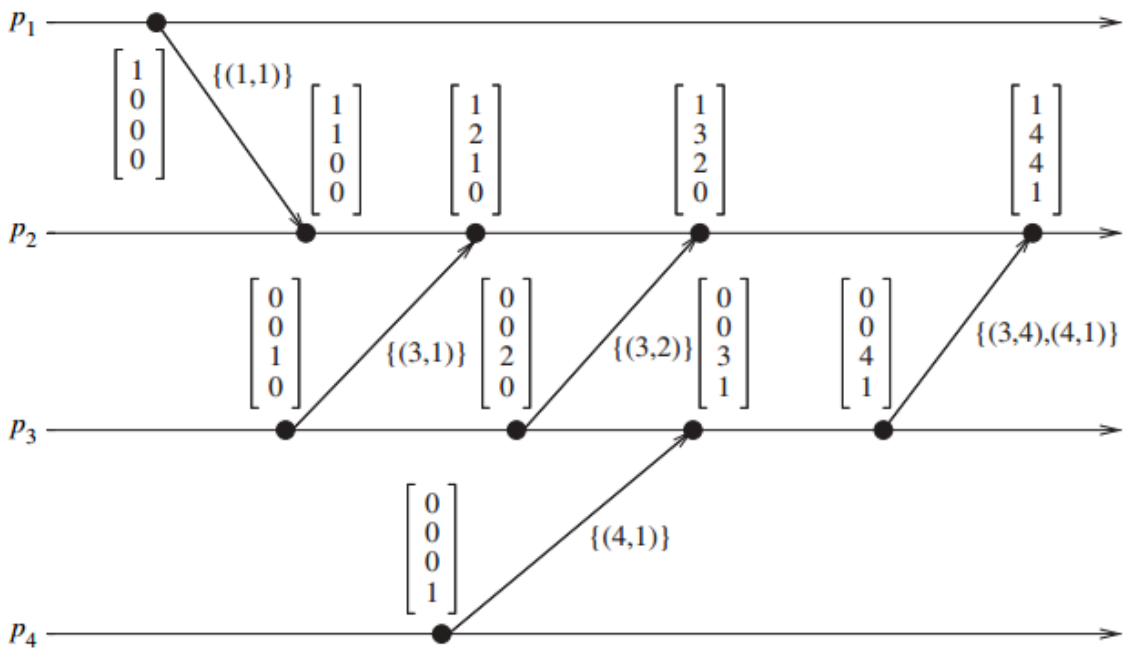



## 第二部分：向量时钟的优化 (SK 算法)

- SK 算法优化向量时钟的存储时间复杂度

**背景：** 如果进程很多（如1000个），每次发消息都带一个长度1000的向量，开销太大。
**Singhal-Kshemkalyani (SK) 技术 (Slide 23-28):**

**原理：**
只发送**“自从上次给你发消息以来，我这边发生变化的部分”**。




**实现机制 (Slide 25):**
每个进程 $P_i$ 需要维护两个辅助数组：

1. $LS_i[1..n]$ (**Last Sent**): $LS_i[j]$ 记录了上次我给进程 $P_j$ 发消息时，我向量里第 $i$ 位的值是多少。
2. $LU_i[1..n]$ (**Last Update**): $LU_i[j]$ 记录了我的向量里第 $j$ 位最近一次被更新时的值。

**发送逻辑 (Slide 26):**
当 $P_i$ 要发给 $P_j$ 时，不需要发整个 $VT$，只需要遍历 $k$，检查：

- **条件：** $LS_i[j] < LU_i[k]$ ?

- **含义：** “上次我发给你之后，第 $k$ 个进程的状态有没有更新过？”
- 如果更新过，就把 $(k, VT_i[k])$ 作为一个元组放进消息里。

**接收逻辑：** 接收方只更新收到的那些 $(k, value)$ 对应位置的最大值。

---

## 第三部分：虚拟时间 (Virtual Time) —— 重中之重

这部分内容很抽象，核心是 **Time Warp (时间这一刻/时间隧道) 机制**。它是基于**乐观 (Optimistic)** 假设的：假设消息永远按顺序到达，如果有迟到的消息（冲突），再回滚。

### 1. 核心概念

- **GVT (Global Virtual Time):** 所有进程中最小的那个时间，它是进度的底线，小于GVT的状态可以被垃圾回收。
- **Local Control (本地控制机制):**
  - 每个进程没有全局钟，只有**本地虚拟时钟 (LVT)**。
  - 处理消息时，LVT 直接跳到消息的时间戳。
- **虚拟时间的消息**
  - 发送进程 PID
  - 接收进程 PID
  - 发送时间（用于 **Output Queue** 的管理。当发送者回滚时，它通过检查这个时间来决定哪些消息需要被撤销）
  - 接收时间（接收者收到消息后，会把消息按这个时间插到 **Input Queue** 里。接收者的 Local Control 就是靠这个时间来决定要把时钟拨到多少）

### 2. 进程的数据结构 (Slide 48) —— 必背

为了支持“后悔药”（回滚），进程必须保存历史：

- **Input Queue:** 存所有收到的消息（按时间排序）。
- **Output Queue:** 存所有发出的消息的**负副本 (Antimessage)**。
- **State Queue:** 存进程历史状态的快照（用于恢复）。

### 3. 回滚机制 (Rollback) 与 Anti-messages —— 核心难点

**场景：**
进程 $P$ 当前时间是 100。突然收到一条时间戳为 90 的消息（迟到了）。
这就违反了因果序（不能先做100的事，再做90的事）。

即如果消息时间戳 < 接收方本地时间，则接收方必须进行回滚。

**处理步骤 (Slide 50):**

1. **恢复状态：** $P$ 去 State Queue 找时间小于 90 的最近状态（比如 89），把自己恢复成那样。
2. **取消影响 (Antimessages):**
   - 在 90 到 100 这段时间里，$P$ 可能已经给别的进程（比如 $Q$）发了消息。这些消息是“错误”的幻影。
   - 为了撤销这些消息，$P$ 从 **Output Queue** 中拿出对应的 **Antimessage**（负消息），发送给 $Q$。

**什么是 Antimessage (反消息)?**

- 它和原消息内容一样，只是符号相反（逻辑上的负号）。
- **湮灭 (Annihilation):** 当一个正消息和一个负消息在队列中同时出现时，它们会**互相抵消，直接消失。**

**接收端的反应 (Slide 51):**
当 $Q$ 收到 $P$ 发来的 Anti-message 时：

- **情况1（幸运）：** 原消息还在 $Q$ 的输入队列里没处理。$\rightarrow$ 正负抵消，直接删掉，无事发生。
- **情况2（不幸）：** 原消息已经被 $Q$ 处理了。$\rightarrow$ $Q$ 也必须**回滚**（因为它基于错误的信息做了计算），并可能继续向其他人发 Anti-message。这叫**级联回滚 (Domino Effect)**，但在Time Warp中是可以收敛的。
- **情况3（幸运）：** 原消息还没到达。$\rightarrow$ $Q$ 被加入输入队列。接收方执行输入队列时将跳过反消息。

---

## 第四部分：物理时钟对齐 (NTP)

这部分考点主要是**计算时钟偏移和往返延迟的 Offset (偏移量)。**

**原理 (Slide 61-66):**
通过**往返时间 (Round-Trip)** 来估算。
假设 A 和 B 对时：

1. A 发送请求 ($T_{i-3}$)
2. B 收到请求 ($T_{i-2}$)
3. B 发送响应 ($T_{i-1}$)
4. A 收到响应 ($T_i$)

**两个核心公式 (Slide 66 公式 6 和 7):**

1. **Offset ($O_i$):** B 相对于 A 快了多少？
   $O_i = \frac{(T_{i-2}-T_{i-3})+(T_{i-1}-T_i)}{2}$
   - *记忆技巧：*（去程的差 + 回程的差) / 2。注意 $T_{i-2}$ 和 $T_{i-1}$ 是 B 的时间，放前面减 A 的时间（或者反过来，取决于题目问谁相对于谁）。
2. **Delay ($D_i$):** 这一趟网络传输一共花了多久？
   $D_i = (T_i - T_{i-3}) - (T_{i-1} - T_{i-2})$
   - *记忆技巧：*（总耗时) - (B处理耗时)。

---

# 第四章：全局一致快照算法

- 分布式系统中三种事件：进程内部事件、消息发送事件、消息接收事件
- **全局一致状态**：进程状态和信道状态的集合
  - **定义**：进程本地状态（寄存器状态、内存、变量等） + 进程之间通信状态（消息发送和接收）；满足因果性：一个事件被接收，则其一定被发送

---

# 第一部分：Chandy-Lamport (CL) 算法原理

这是所有快照算法的祖师爷。它的前提是：**信道必须是 FIFO（先进先出）的。**

## 1. 核心思想：Marker（标记）

把数据流想象成水流，CL算法就是往水里扔一个红色的浮标（**Marker**）。

- Marker 之前流过去的消息，属于“过去”（快照前）。
- Marker 之后流过来的消息，属于“未来”（快照后）。

以此记录进程的内存状态 + 信道里正在传输的消息（Channel State），信道转态就是“我拍了照”之后，但在“收到你的Marker”之前，这段时间里收到的消息。

## 2. 算法流程（两个规则）

- **发送规则 (Marker Sending Rule) - Slide 14:**
  - 当一个进程 $P$ 决定拍快照时：
    1. 先立刻记录自己的**本地状态**。
    2. 然后立刻向**所有**出边的信道发送 **Marker**。
    3. （Marker 就像一道闸门，把前后的消息隔开了）。
- **接收规则 (Marker Receiving Rule) - Slide 14:**
  - 当进程 $Q$ 从信道 $C$ 收到一个 Marker 时：
    - **情况 A（第一次见到 Marker）：** 说明 $Q$ 还没拍快照。
      1. $Q$ 立刻记录自己的**本地状态**。
      2. 将信道 $C$ 的状态记录为**空**（因为 Marker 是这通道的第一条分界线）。
      3. $Q$ 向自己所有的出边信道发送 Marker（把接力棒传下去）。
    - **情况 B（已经拍过快照了）：** 说明 $Q$ 早就拍完了，但现在才从信道 $C$ 收到 Marker。
      1. 这说明：在 $Q$ 拍完快照之后、收到这个 Marker 之前，信道 $C$ 里可能有别的消息流过来了。
      2. **动作：** 记录**信道状态**。即：把“$Q$ 拍快照时刻”到“收到 Marker 时刻”之间，从信道 $C$ 收到的所有消息，都记下来。

---

# 第二部分：Spezialetti-Kearns (SK) 算法原理

**背景：** CL 算法假设只有一个发起者。如果系统中**同时有多个进程**（并发）都想发起快照，CL 算法会产生多个独立的快照，效率低且浪费带宽。

**SK 算法的核心 (Slide 17-19)： 合并 (Merge)。**

1. **染色与区域 (Regions):**
   - 每个发起的进程都有一个 ID。Marker 上会带着发起者的 ID。
   - 当进程收到 Marker 时，如果 ID 和自己记录的一样，说明大家是“一家人”（同一个 Region）。
   - 如果 ID 不一样，说明是“隔壁阵营”发起的快照。这时候不重复拍，而是记录下**边界 (Border)**。
2. **生成树合并:**

- 快照过程会隐式地形成一棵生成树。
- SK 算法让不同的发起者交换信息，最终把大家拍的局部照片**拼成**一张完整的全局大合照，而不是每个人都去拼凑一遍。

**一句话总结 SK：** 它是 CL 的多发起者优化版，通过维护“区域 ID”来避免重复工作，并高效合并快照。

---

## 第三部分：画图与判断（重难点：切面与 Zigzag Path）

考试肯定会给你一张时空图（Space-Time Diagram），问你：

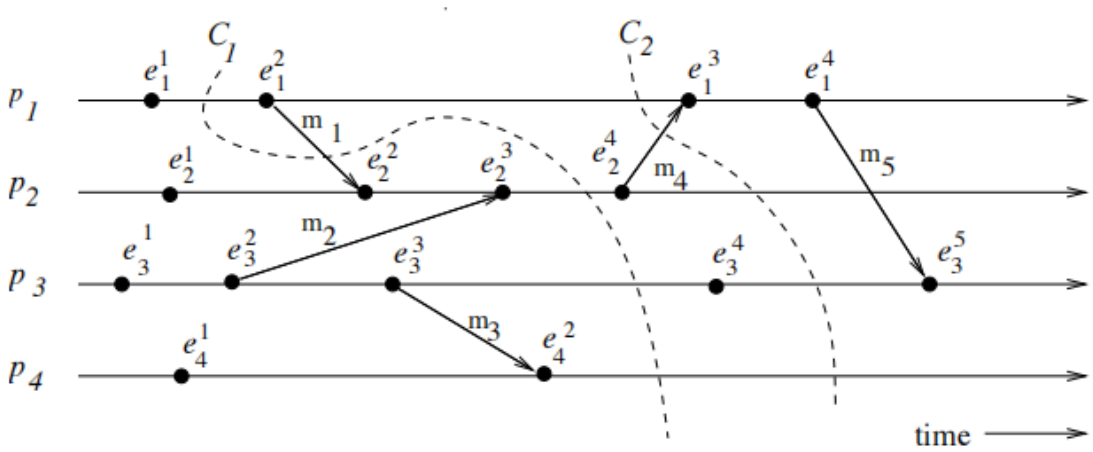
1. 画一条线，看是不是全局一致的？
2. 给几个检查点（Checkpoints），问它们能不能组成一个一致的快照？

### 方法一：切面法 (Cuts) —— 最直观，看图必用

**原理 (Slide 9, 27):**
在图上画一条贯穿所有进程的线（Cut）。

- **判断标准： 不允许“从未来发往过去”的消息。**
- **具体操作：**
  1. 检查切面线切断的所有消息箭头。
  2. 如果箭头的**尾巴（发送）**在切面右边（未来），而**头（接收）**在切面左边（过去）。
  3. **这就是不一致的！**（不可能还没发就收到了）。
  4. 反之，如果所有切断的箭头都是“左发右收”（发了还没收），那是允许的（信道状态）。

**看 Slide 10 的图：**




- $C_1$ **(左边的曲线):** 切断了消息 $m_1$。$m_1$ 是从右边发出的（$P_2$），但在左边被接收（$P_1$）。**不一致！**
- $C_2$ **(右边的曲线):** 切断了消息 $m_4$。$m_4$ 是左发右收。**一致！**

### 方法二：Zigzag Path (Z-Path) —— 判定检查点集合

**场景：** 题目不再让你画线，而是给了你几个点：$C_{1,1}$（进程1的第1个存档），$C_{2,2}$... 问这些点能不能凑一对？

**核心理论 (Slide 36, 39):**

- **Z-Path 定义：** 一条消息链。但它比因果链更宽容。允许消息 $m_1$ 收到后，$m_2$ 在**同一个检查点区间内**发送（哪怕 $m_2$ 发送时间早于 $m_1$ 接收时间）。

- *口诀：* 只要在同一个区间里，就可以“时光倒流”去接力。
- **Z-Cycle (Z环):** 如果从检查点 $A$ 出发，顺着 Z-Path 走了一圈又回到了 $A$。
- **判定法则 (Slide 39, 41):**
  1. **如果检查点 $C$ 在一个 Z-Cycle 中** $\Longrightarrow$ 它**永远不可能**成为任何一致快照的一部分。（它是个坏点）。
  2. **如果两个检查点 $A$ 和 $B$ 之间有 Z-Path** $\Longrightarrow$ 它们**不能同时**出现在同一个一致快照里。
- 结论：
  - 全局一致性快照的充要条件：两个检查点之间没有 zigzag 路径
  - 全局一致性快照的充要条件：一个检查点能够成为一致性快照的一部分，当且仅当该检查点不属于 zigzag 环路（ zigcycle ）
  - 全局一致性快照的必要条件：两个检查点之间没有因果路径，即：全局一致性快照检查点之间没有因果路径，但没有因果路径不能作为判断的充分条件

**怎么做题？ （以 Slide 34, 37 为例）**

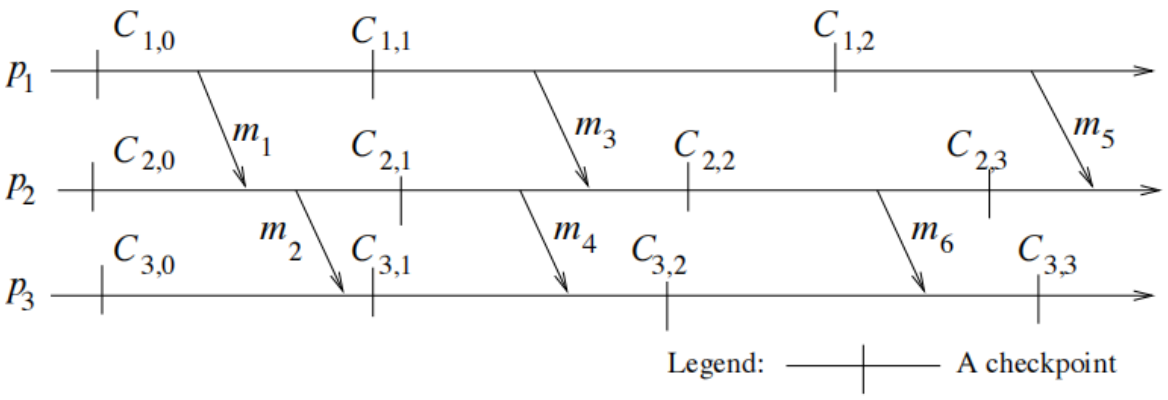



Figure 4.2: An Illustration of zigzag paths.

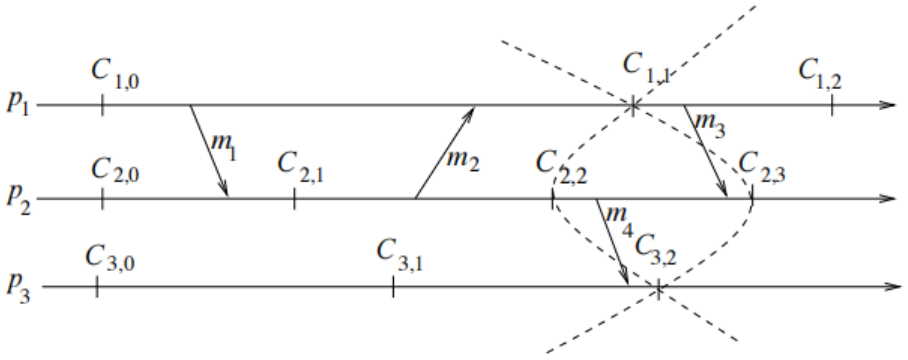



Figure 4.3: A zigzag cycle, inconsistent snapshot, and consistent snapshot.

1. **找 Z-path：**
   - 看图 4.2 (Slide 34)：从 $C_{1,1}$ 开始，消息 $m_3$ 传给 $P_2$。在 $P_2$ 的同一区间（$C_{2,1}$ 到 $C_{2,2}$ 之间），有一条消息 $m_4$ 发给了 $P_3$。
   - 注意！虽然 $m_4$ 是在 $m_3$ 到达之前就发出的，但因为它们在**同一个区间**，所以构成了 Z-Path 连接。
   - 路径：$C_{1,1} \xrightarrow{m_3} P_2 \xrightarrow{\text{同区间}} m_4 \rightarrow P_3$ (到达了 $C_{3,2}$ 之前)。

- **结论：** $C_{1,1}$ 和 $C_{3,2}$ 之间有 Z-Path，所以它俩**不能**在同一个快照里。

2. **找 Z-Cycle (最坏的情况):**
   - 看图 4.3 (Slide 37)：看 $C_{2,1}$。
   - 消息 $m_2$ 传出去给 $P_1$。
   - $P_1$ 在同一区间发回消息 $m_1$ 给 $P_2$。
   - $m_1$ 在 $C_{2,1}$ 之前到达。
   - 形成环路：$C_{2,1} \to m_2 \to P_1 \to m_1 \to C_{2,1}$。
   - **结论：** $C_{2,1}$ 在 Z-Cycle 上，它是个废点，**没有任何**一致快照能包含它。

---

# 第六章：术语和基本算法

## 一、 基础术语与分类 (Classifications and Basic Concepts)

梳理基础术语，有助于我们理解各种算法的设计目标和约束，不是核心重点

1. **控制算法 (Control Algorithm) vs 应用执行 (Application Execution) (Slide 5-6):**
   - **应用执行：** 是分布式系统在做“正事”，比如处理用户请求、计算数据。
   - **控制算法：** 是“幕后英雄”，负责协调、管理、维护系统正常运行，比如：
     - **ST (Spanning Tree)**：建立通信树。
     - **MIS (Maximal Independent Set)**：找到一组互不相邻的节点。
     - **CDS (Connected Dominating Set)**：找到一组节点，它们自己能连成一片，并且能“覆盖”到网络里的所有其他节点。
     - **全局状态检测**：比如死锁、终止检测、快照。
     - **容错 (Checkpointing)**。
   - **关系：** 控制算法是“叠加”在应用执行之上的，但其通信（send/receive/internal events）对应用层是透明的（就像操作系统内核的工作）。
2. **集中式 (Centralized) vs 分布式 (Distributed) 算法 (Slide 5-6):**
   - **集中式：** 有一个明确的“总指挥”（比如根节点）。优点是简单，缺点是总指挥是单点故障。
   - **分布式：** 没有一个绝对中心，所有节点地位相对平等，共同协作完成任务。设计难度大，但更鲁棒。
3. **对称 (Symmetric) vs 非对称 (Asymmetric) 算法 (Slide 6):**
   - **对称：** 所有节点执行的代码逻辑完全一样（比如 LCR 选主算法，虽然 ID 不同，但代码逻辑一样）。
   - **非对称：** 节点角色不同，执行的代码或行为逻辑也不同（比如 Leader Election 里，Leader 的行为和普通节点的行为就不同）。
4. **匿名 (Anonymous) vs 统一 (Uniform) vs 自适应 (Adaptive) 算法 (Slide 7):**
   - **匿名：** 算法运行中**不使用**节点的 ID 来做决策。这在环形网络选主时很难做到确定性。
   - **统一：** 算法代码**不能**直接用到节点总数 $N$。这有利于提高可伸缩性，节点增减对算法影响小。
   - **自适应：** 算法的复杂度和参与的节点数 $k$ 有关，而不是总节点数 $N$。例如，互斥算法的冲突开销可能只跟当前在临界区外等待的节点数 $k$ 有关。
5. **确定性 (Deterministic) vs 非确定性 (Nondeterministic) 执行 (Slide 8):**

- **确定性：** 即使网络延迟不同，事件的相对顺序（因果序）也是固定的。调试相对容易。
- **非确定性：** 存在“不可靠接收”（不知道消息从哪儿来）或者网络延迟、CPU调度不确定，导致每次运行的事件顺序可能不同。调试困难。

6. **执行抑制/冻结 (Execution Inhibition / Freezing) (Slide 9):**
   - 某些控制算法（如同步算法）为了保证正确性，可能会**暂停**部分进程的正常执行，直到某个条件满足（比如收到所有邻居的确认）。
7. **同步 (Synchronous) vs 异步 (Asynchronous) 系统 (Slide 10):**
   - **同步：** 有界消息延迟，有界时钟漂移，按轮次执行。
   - **异步：** 无界延迟，时钟漂移不确定，事件驱动。大部分分布式系统本质上是异步的。
8. **On-line vs. Off-line (控制) 算法 (Slide 10):**
   - **On-line：** 边跑边收集数据（如运行中的网络监控）。
   - **Off-line：** 需要先收集好所有需要的数据，再开始执行（如离线日志分析）。
9. **无等待 (Wait-free) 算法 (Slide 11):**
   - **目标：** 即使有进程失败（最多 $n-1$ 个），算法也能保证**有限步内**完成。
   - **特点：** 非常鲁棒，但通常开销也大。
   - **通信通道：** FIFO 或 Non-FIFO。
10. **进程/链路故障模型 (Process/Link Failures) (Slide 12-13):**
    - **进程故障：**
      - **Fail-stop:** 停止运行，且**其他进程能感知到。**
      - **Crash:** 停止运行，但**其他进程可能不知道。**
      - **Omission:** 漏收/漏发消息。
      - **Byzantine:** 行为**任意**，可能伪造消息，非常难处理。
    - **链路故障：** 链路挂断（Crash）、漏发（Omission）、甚至伪造消息（Byzantine）。

---

## 二、 经典分布式图算法 (Slide 16-38)

**同步算法和异步算法**：

- 同步算法和异步算法的区别主要在于**节点间是否需要统一的全局时钟 。**同步算法通过全局时钟以轮次推进， **节点步调一致（消息发送，消息接收，信息更新），消息在固定时间内到达**（如果没有送达，那就是节点故障，不属于同步系统了），设计简单但对同步依赖强；而异步算法没有全局时钟， **节点基于消息触发独立运行（消息的发送和接收没有固定时间）** ，但设计和分析更复杂，可能增加消息冗余。

# Basic Distributed Graph Algorithms: Listing

- Sync 1-initiator ST (flooding)
- Async 1-initiator ST (flooding)
- Async conc-initiator ST (flooding)
- Async DFS ST
- Broadcast & convergecast on tree
- Sync 1-source shortest path
- Distance Vector Routing
- Async 1-source shortest path
- All sources shortest path: Floyd-Warshall
- Sync, async constrained flooding
- MST, sync
- MST, async
- Synchronizers: simple, $\alpha$, $\beta$, $\gamma$
- MIS, async, randomized
- CDS
- Compact routing tables
- Leader election: LCR algorithm
- Dynamic object replication

## 1. 生成树 (Spanning Tree - ST)

**任务：** 在一个网络图里，选一个根节点，找到连接所有节点的最少边集合，形成一棵树。形成一个可知的稳定的通信网络。

- **同步 1-initiator ST (Flooding) (Slide 17-19):**

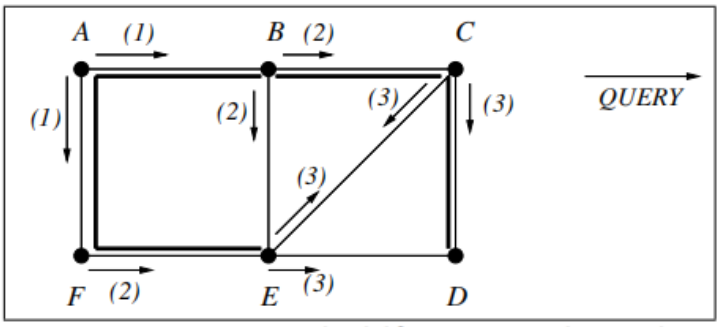
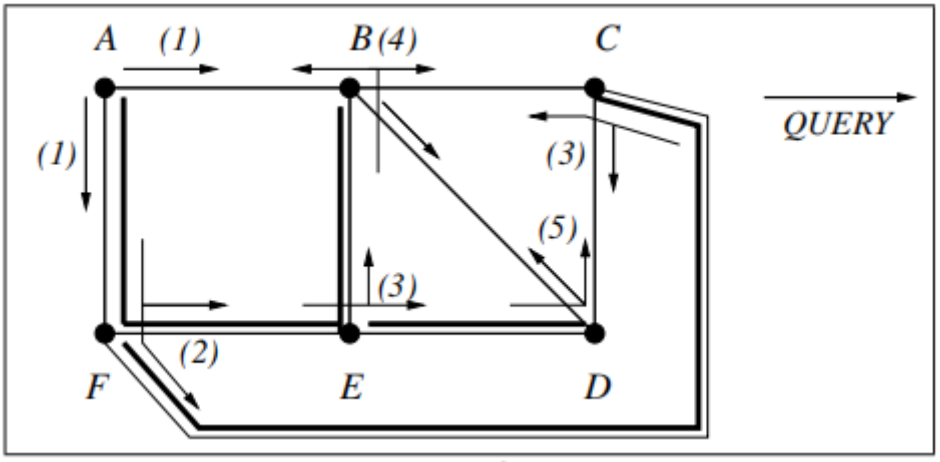


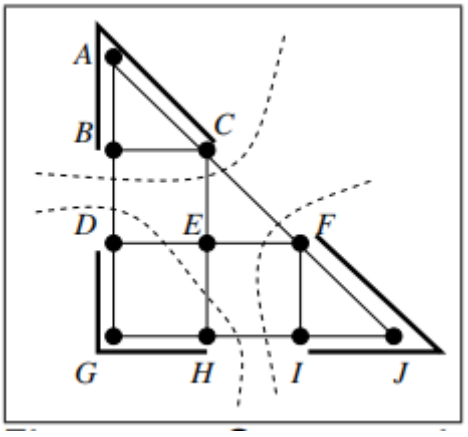

Figure 5.2: Tree in boldface; round numbers of QUERY are labeled

  - **原理：** 类似 CL 算法里的 Marker，用 `QUERY` 消息向外"探测"。
  - **过程：**
    1. **Root 发起：** `QUERY` 广播。
    2. **普通节点收到：** 第一次收到 `QUERY`，认发来的节点为 `parent`，记录深度，然后把 `QUERY` **转发**给其他邻居（除了父节点）。
    3. **收到多个 QUERY?** 随机选一个（或者按 ID 选）认作父亲，其他的则被忽略（这些来自"兄弟"的 QUERY）。
    4. **终止：** 当一个节点的深度到达图的直径（最长路径）时，它就不再转发了。或者当它发现自己所有的邻居都已"访问过"（visited=1），也就停止了。
  - **特点：** 生成的是 **BFS 树**（层次清晰，每一轮扩散一层）。
- **异步 1-initiator ST (Flooding) (Slide 20-23):**




- **原理：** 同样是 `QUERY` 消息，但更灵活。
- **过程：**
  1. **Root 发起：** `QUERY` 广播。
  2. **普通节点收到：**
     - 如果第一次收到 `QUERY` (parent is null)：认发来的节点为 `parent`，然后把 `QUERY` 转发给除了父节点之外的所有邻居。
     - 如果不是第一次收到`QUERY`，直接发送 `REJECT`给发方，并将其加入`Unrelated`
     - 如果收到了 `ACCEPT`：把它加入 `Children`。
     - 如果收到了 `REJECT`：把它加入 `Unrelated`。
  3. **终止条件：** 当一个节点发现自己的 `Children` 加上 `Unrelated` 列表包含了**所有**邻居（除了父节点）时，它就认为自己该做的都做完了。这时只需向`parent`发送`ACCEPT`。等到发起者收敛时，整个程序也收敛了
- **特点：** 树的形状不确定，取决于消息到达顺序。`REJECT` 消息用于拒绝非树边（Cross-edge），避免形成环。

- **异步并发发起者 (Concurrent Initiators) (Slide 24):**




- **问题：** 如果多个节点同时想当 Root 发起 ST。
- **选项 1 (Merge partial STs)：** 各自跑，最后把结果拼起来。难点在于如何根据本地知识判断合并，容易出错。
- **选项 2 (Select one Root)：** 允许一个 ST 计算继续，压制其他。通常选择 ID 最大的作为最终 Root。
  - 最终还是等价于 J 发起的生成树
  - 在具体执行时，与单异步相似，只是多了rootID的比较以及关系的打破与重建
- **终止：** 只有最终 Root 能判断整个 ST 是否建立完毕。

- **异步DFS生成树**
  - 与正常异步完全类似，只是在发送消息时不再广播，而是逐一发送（收到当前节点的回信时，再query下一节点）

---

## 2. 广播与汇聚 (Broadcast & Convergecast) (Slide 29-30)

- **Broadcast：** 从根节点向所有叶子节点传播信息。
  - **BC1 (Root):** 直接发给所有孩子。
  - **BC2 (Non-root):** 收到父节点的信息，复制一份，发给所有孩子。
- **Convergecast：** 从叶子节点向根节点收集信息，并在根节点聚合。
  - **CVC1 (Leaf):** 把自己的报告发给父节点。
  - **CVC2 (Non-root):** 收到所有孩子（及自身）的报告后，合并成一个报告发给父节点。
  - **CVC3 (Root):** 收到所有孩子的报告后，执行全局函数（如求和、最大值、选 Leader）。
- **应用：** 选 Leader、算全局最小值/最大值、做快照。
- **特征：** 消息复杂度是 $O(n-1)$，即每条边传输一次消息（广播一次，汇聚一次）。

---

## 3. 最短路径 (Shortest Path)

**任务：** 在一个网络图里，选一个根节点，最小化到其他节点的开销

- **同步 Bellman-Ford (Slide 32-33):**
  - **原理：** 按轮次迭代，每轮广播一次自己当前已知的最短距离。
  - **过程：** 知道到邻居的距离。下一轮，通过邻居的距离 + 邻居到自己的边权重，更新自己的距离。
    - 一开始只有源节点距离是0，其他节点距离都是$\infty$
    - 所有节点向邻居广播自己到A节点的距离。然后接收节点根据到邻居的距离和邻居到A的距离，确定最近的距离
    - 迭代上一步直至算法收敛
  - **终止：** 跑 $N-1$ 轮（或者直到某轮所有距离都没更新，$N$个节点更新 $N-1$轮一定会收敛）。
  - **特点：** 保证收敛，但同步开销大。
- **距离向量路由 (Distance Vector Routing) (Slide 33):**
  - **原理：**向量无穷版的 Bellman-Ford。
  - **特点：**在真实的互联网中，所有节点的互通信都是必要的，所以每个节点维护的不仅仅是到源节点的距离，而是到所有节点的最短路径存在LENGTH[]里面，PARENT[]存放的是去对应ID节点需要转发的节点
  - **过程：** 谁的距离更新了，就告诉所有邻居。邻居收到后，看能不能优化自己的路径。
  - **缺点：** 容易出现 **Count-to-Infinity** 问题（比如网络环路导致距离信息不断增加，直到溢出）。两个节点互相路由到无穷大（比如C和B到A需要互相经过对方，导致路由距离爆炸）
- **异步 Bellman-Ford (Slide 34-35):**
  - **原理：** 整体思路与同步Bellman-Ford是一致的，还是利用三角不等式更新距离。只是因为异步系统无法保证通信到达的时间，所以收到消息就更新，发生变化就广播，没有确定的收敛边界，依靠所有消息静默自然停止。

- **过程：** 只要收到邻居更新的距离信息，就立刻更新自己的，然后把更新后的信息再发出去。
- **问题：** 网络震荡时，可以会出现更新震荡，重复出现（*更优解覆盖次优解*）。

- **Floyd-Warshall (All-Pairs Shortest Paths) (Slide 36-37):**
  - **目标：** 计算图中**任意两点**之间的最短路径。
  - **集中式原理：** 三重循环。核心是考虑中间节点 `pivot`。对于任意两点 `s` 和 `t`，看能否通过 `pivot` 走一条更短的路：$path(s,t)$ vs $path(s,\text{pivot}) + path(\text{pivot},t)$。
  - **分布式困境：** $path(\text{pivot},t)$对于节点$s$是未知的，想要让s知道这一信息，需要让pivot节点发送给所有它可达的节点，它手上现有的数据，但是盲目的泛洪浪费了太多的带宽，所以就引入了汇聚树（Sink Tree）。
  - **分布式实现：** 简单来说就是通过汇聚树的反向索引能力，只向最短路径上的邻居发送pivot数据信息，即节点 $i$ 维护自己知道的 $s \to j$ 的长度 `LEN[j]` 和路径的第一个节点 `PARENT[j]`。再通过“Sink Tree”同步，得到$path(\text{pivot},t)$
  - **其他数据传播方式**：受限泛洪，上面我们讲到盲目的泛洪会使消息在网络里无限死循环和爆炸，所以在异步系统里我们可以通过版本号鉴别消息的时效，同步系统我们可以通过有限的迭代轮次实现受限的泛洪策略。

---

## 4. 最小生成树 (MST)

- **基本概念：** 在连通图中，找到一个子集，它连接了所有节点，并且总边权重最小。
- **Kruskal vs Prim：** Kruskal 是先排序所有边，贪婪选择。Prim 是从一个节点开始，不断扩展。这两个都是集中式的系统中的MST算法
- **GHS (Gallagher-Humblet-Spira) 算法 (Slide 48-53)：**
  - **原理：** 分布式 Kruskal。先在每个连通分量内找到最小权重的**出边 (MWOE - Minimum Weight Outgoing Edge)**。
  - **过程：**
    1. **Leader Election:** 每个连通分量内部选个 Leader。
    2. **Identify MWOE:** Leader 广播 `SEARCH_MWOE`，向外探寻。
    3. **Convergecast:** 各个节点汇报自己找到的 MWOE。
    4. **Merge:** Leader 收到报告后，选择全局最小的 MWOE，形成新的连通分量，并选出新的 Leader。（选新的Leader是为了防止联通后存在多个Leader导致脑裂）
  - **特点：** 同步算法，按轮次进行，每次大约合并一半的组件，效率很高（$logn$ 轮）。
  - **异步实现的困境：** Slide 53
    - MST有一个原则是不新增当前连通分量内的边，所以需要确定待拓展的新边的入节点不属于当前连通分量。
    - 这一点在同步系统内可以简单的判断，
    - 但是在异步异步系统中，新Leader的选举可能晚于其他集群的链接，这会导致部分节点的Leader在运行时不能被及时更新，导致算法出错。甚至形成通信环路
  - **异步实现方法：** 用$\gamma$**同步器**在异步系统上模拟同步行为 Slide 54
    - 工程方法不要求掌握，简单来说就是引入版本号（异步算法常用的操作），高版本归并需要等待低版本推进。

## 5. 同步器 (Synchronizers) (Slide 55-57)

- **目的：** 在异步系统上模拟同步行为。
- **原理：** 通过额外的消息和检查，强制大家按轮次来。
- **简单同步器：** 每轮发个消息，收到邻居确认后进入下一轮。
- $\alpha$ **同步器：** 引入“Safe”信号。只有当所有邻居都确认自己安全了，我才能进入下一轮。
- $\beta$ **同步器：** 建立生成树，通过**汇聚 (Convergecast)** 确定全局安全，再用**广播 (Broadcast)** 通知大家进入下一轮。
- $\gamma$ **同步器：** 分区（Cluster），先在区内用 $\beta$，再用 $\alpha$ 处理区间的通信。

---

## 6. 最大独立集 (Maximal Independent Set - MIS) (Slide 64-66)

- **定义：** 找到节点集合 $S$，满足：
  1. **独立性：** 集合 $S$ 中的任意两个节点之间**没有边**。
  2. **最大性：** 集合 $S$ 不能再添加任何节点而不破坏独立性。（比它大的集合就不是独立的了）。
     - 最大独立集的大小不唯一
     - 规模最大的MIS称为极大独立集
     - 极大独立集，集合的大小唯一，元素不唯一
- **Luby's Randomized Algorithm：**
  - **原理：** 随机选，冲突就淘汰。
  - **过程：**
    1. 大家随机选个号。
    2. **Winner：** 邻居中号最小的胜出。
    3. **Eliminated：** 胜出的节点加入 MIS，它的邻居们会被淘汰。
  - **特点：** 异步，随机，能保证找到一个 MIS，但不是最大的（Maximum）。
- **示例**

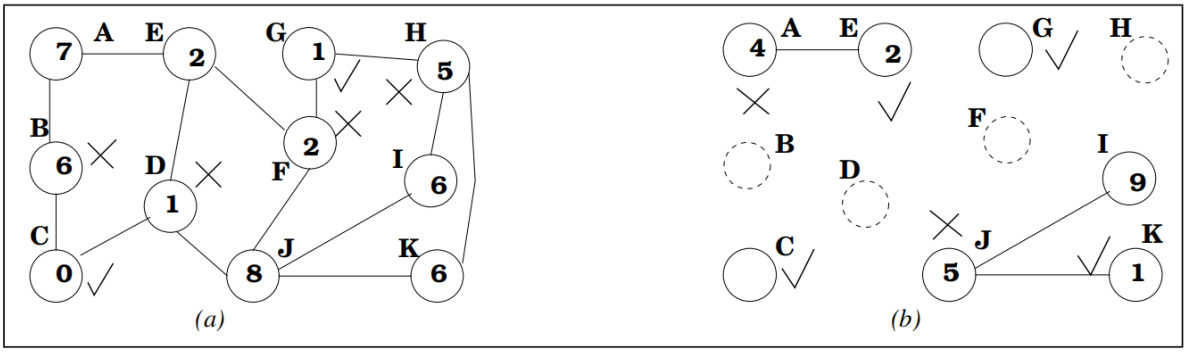



Figure 5.12: (a) Winners and losers in round 1. (b) Winners up to round 1, losers in round 2.

Third round: I is winner. MIS$=\{C, E, G, I, K\}$.

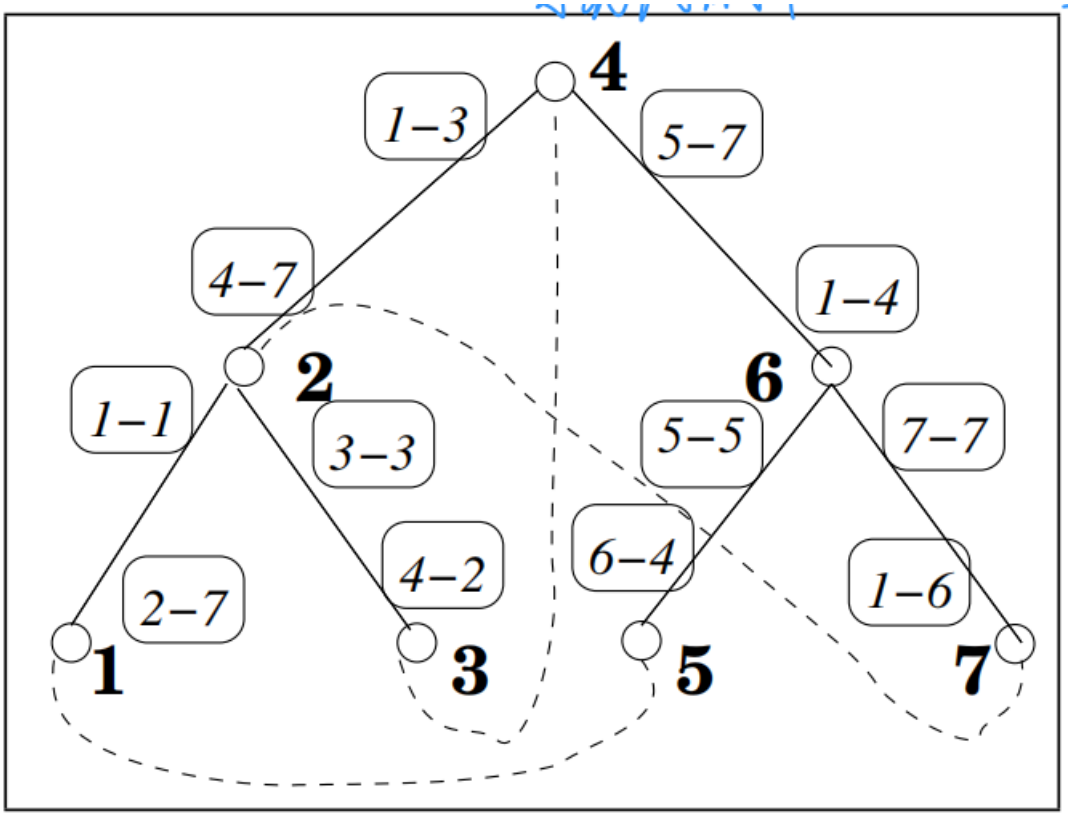
Note: $\{A, C, G, J\}$ is a smaller MIS.

## 7. 连接支配集 (Connected Dominating Set - CDS) (Slide 68)

- **支配集 (Dominating Set)：** 集合 $S$ 中的节点，能“连接”到 $S$ 之外的所有节点（即 $S$ 外的每个节点，至少有一个邻居在 $S$ 里）。
- **连接支配集 (CDS)：** 支配集 $S$ 里的节点，自己还要能连成一片（不能是分散的）。
- **方法：**使用ST裁剪叶子；使用MIS增加连边
- **应用：** 充当网络的“骨干”，便于广播。
- **困境：**找最小CDS是NP完全问题

---

## 8. 紧凑路由表 (Compact Routing Tables) (Slide 69-71)

- **问题：** 在大型网络（如互联网）中，每个节点存储所有节点路由信息会占用巨大空间。
- **目标：** 用尽可能小的路由表，依然能找到路径。
- **方法：**
  - **层次路由**：IPV4
  - **树标记 (Tree Labeling)：** 给节点和边打标签，利用树的结构来编码路由信息。
  - 

  - **区间路由 (Interval Routing)：** 给每个节点分配一个 Label，给每条边分配一个 Label 集合。路径就是通过 Label 的匹配来找到。（也就是利用了上面的树标记方法）
    - 所有邻边的编号集合无交集（无冗余）
    - 所有邻边的Label集合取并集+自身的编号=全集（全图可达）
  - **前缀路由 (Prefix Routing)：** 类似 IP 地址，找标签是目标地址最长前缀的边（前缀最匹配）。
- **Stretch Factor：** 衡量路由路径长度是实际最短路径的多少倍。

  $\text{Stretch} = \max_{i,j} \frac{\text{紧凑路由算法算出的路径长度}}{\text{真实的最短路径长度}}$

---

## 9. 领导者选举 (Leader Election) (Slide 72-74)

- **定义：** 让所有进程就一个“首领”达成一致。
- **LCR (LeLann-Chang-Roberts) 算法 (Slide 73)：**
  - **拓扑：** 异步无向环形网络。
  - **过程：** 向右边的邻居传 ID（自己的或者更大的ID消息），大的淘汰小的，最后 ID 最大的获胜。
  - **注意：** 在匿名环（不知道自己是谁，也不知道总共有多少人）上，没有确定性算法。
- [Hirschberg-Sinclair Algorithm：]
  - **原理：** 在有向环上**二分查找**。
  - **过程：** 消息向左右两边扩散，找最大 ID。每轮确定一些“候选 Leader”。最终只有一个能留下来。
  - **优点：** 比 LCR 消息复杂度低。

---

## 10. 对象复制 (Object Replication) (Slide 75-81)

- **问题：** 如何在网络中放置多个数据副本，使得读写操作的成本最低？
- **核心思想：**
  - **读操作：** 找到离自己最近的副本去读。
  - **写操作：** 写给离自己最近的副本，然后**向其所在的集合传播**（Propagate in R）。
  - **R (Replicas)：** 存放副本的节点集合。这个集合会根据读写成本动态调整，像**变形虫**一样扩张或收缩。
- **算法测试：**
  - **Expansion Test:** 如果读成本 > 写成本，把邻居加到 R 里。
  - **Contraction Test:** 如果写成本 > 读成本，把自己从 R 里移除（如果邻居允许）。
  - **Switch Test:** 节点在 R 里待着，还是给邻居腾出位置，看哪个更划算。
- **困境：**
  - 以上规则对于任意图是NP完全问题
  - 所以PPT里讨论了在树形结构下的情形

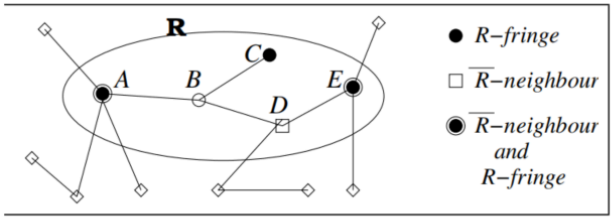



Figure 5.14: Nodes in ellipse belong to $R$.

- $C$ is $R$-fringe
- $A$, $E$ are $R$-fringe and $\overline{R}$-neighbour
- $D$ is $\overline{R}$-neighbour

$\overline{R}$-neighbour: $i \in R$; and has at least one neighbour $j \notin R$.

$R$-fringe: $i \in R$; and has only one neighbour $j \in R$. Thus, $i$ is a leaf in the subgraph of $T$ induced by $R$ and $j$ is parent of $i$.

singleton: $|R| = 1$ and $i \in R$.

- **说明：**
  - 上图定义了R集合里面各个节点的名称
  - Expand Test是针对于neighbour节点，如果邻居读>写就把邻居加入R

- Contraction Test是针对于fringe节点，如果自身写 > 读就退出R，退出前需要确定自己不是唯一副本
- Switch Test是针对于singleton，本质上是对于单副本R集合选取最高频的读+写邻居节点进行置换。

# 第七章：消息序和组通信

## 一、 消息序的基础：FIFO vs CO vs SYNC

首先，我们需要区分三种越来越严格的消息排序：

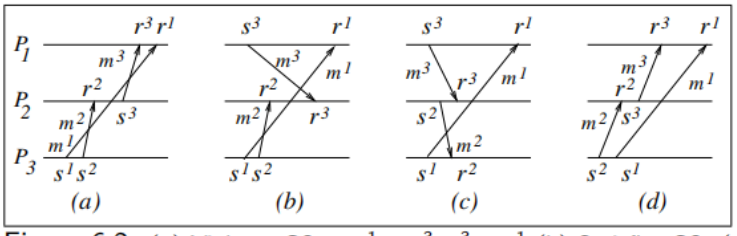
1. **FIFO (先进先出) (Slide 4):**
   - **定义：** 对于**同一对**发送者 $P_i$ 和接收者 $P_j$，如果 $P_i$ 先发了 $m_1$ 后发了 $m_2$，那么 $P_j$ 必须先收 $m_1$ 再收 $m_2$。
   - 即$(s \sim s' \text{ and } r \sim r' \text{ and } s \prec s') \implies r \prec r'$
   - **局限：** 只管两进程之间，不管跨进程的因果关系。
   - **应用：** TCP，non-FIFO上实现FIFO使用序列号和确认id
2. **CO (Causal Order, 因果序) (Slide 5-7): —— 重点**
   - **定义：** 如果发送事件 $send(m_1) \to send(m_2)$（因果先于），那么在**任何**共同的接收者那里，必须先交付 $m_1$ 再交付 $m_2$。
   - 即$(r \sim r' \text{ and } s \prec s') \implies r \prec r'$，发送事件不必是同一进程
   - **图示判断 (Slide 5):**
     - 

Figure 6.2: (a) Violates CO as $s^1 \prec s^3$; $r^3 \prec r^1$ (b) Satisfies CO. (c) Satisfies CO. No send events related by causality. (d) Satisfies CO.
     - 图(a) **违反CO**：先找同一进程内的接收事件$r_3 \prec r_1$，再看发送事件$s_1 \prec s_3$,所以违反了CO
     - 图(b) (c) (d) **满足CO**：(b)相关的发送事件是并发的，（c）没有关联的接收事件，（d）相关的发送事件是并发的。
   - CO的其定义
     - 如果具有相同接受进程的发送事件 $send(m^1) \prec send(m^2)$ 那么一定满足 $deliver(m^1) \prec deliver(m^2)$ .(deliver 表示消息达到缓冲区并且被处理)
     - $s \prec s' \implies \neg(r' \prec r)$ 这个公式的解题角度是先找到有偏序关系的发送事件
     - 从偏序关系的角度考虑，$\{x \in E \mid s \prec x \prec r\}$这个集合必须是一个空集
     - 偏序关系的另一个角度是相同的历史和未来，即$s \prec e \implies \neg(e \prec r)$或者 $e \prec r \implies \neg(s \prec e)$。
3. **SYNC (Synchronous Order) (Slide 10):**
   - **定义：** 就像所有消息都是在一个全局的大钟下，瞬间完成传输。

- **直观判断 (Slide 10):** 如果能把图画得像右边那样，所有的箭头都是**垂直**的（或总是从左向右倾斜，且不交叉），那就是 SYNC。

**包含关系 (Slide 17):**

- $RSC \subset CO \subset FIFO \subset Asynchronous$。

---

## 二、 RSC 与 Crown (皇冠) —— 必考难点

**1. 什么是 RSC (Realizable with Synchronous Communication)?**

- **概念 (Slide 12):** 一个异步执行（Asynchronous Execution），如果能在同步系统上**无死锁**地跑出来，就叫 RSC。
- **前提：** 不可分离线性拓展（Non-separated linear extension）
  - 就是对于满足偏序关系的事件集合里面的任意事件对$\{x \in E \mid s \prec x \prec r\}$这个集合都是一个空集
  - RSC的必要条件就是异步执行的事件集合是不可分离线性拓展的
  - 但是使用这种方法去判断RSC是指数级的复杂度，所以实践中使用皇冠判别法
- **直观理解：** 它的消息模式非常“规矩”，没有那种复杂的“你等我、我等你”的死锁依赖。

**2. 怎么判断 RSC？ （找 Crown）**
**Crown (皇冠)** 是 RSC 的克星。**如果有 Crown，就不是 RSC。**

- **Crown 的定义 (Slide 14):**
  一个大小为 $k$ 的 Crown 是一个序列 $\langle (s^0, r^0), (s^1, r^1), \ldots, (s^{k-1}, r^{k-1}) \rangle$，满足：
  $s^0 \prec r^1, \quad s^1 \prec r^2, \quad \ldots, \quad s^{k-1} \prec r^0$
  - *解释：* 第 0 个消息的发送，发生在第 1 个消息接收**之前**。第 1 个发送在第 2 个接收之前... 最后一个发送在第 0 个接收之前。形成了一个**闭环依赖**。

**3. 给一个例子找 Crown 判断 RSC (Slide 14 图)**

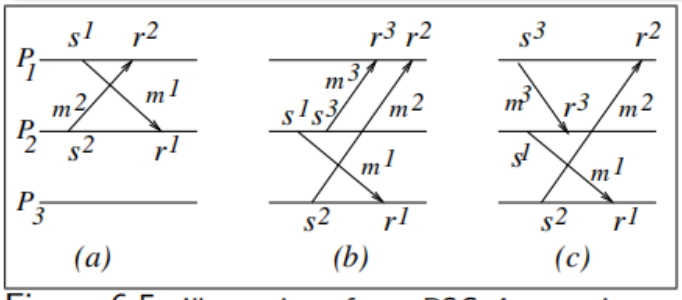



Figure 6.5: Illustration of non-RSC $A$-executions and crowns.
Fig 6.5(a): crown is $\langle (s^1, r^1), (s^2, r^2) \rangle$ as we have $s^1 \prec r^2$ and $s^2 \prec r^1$
Fig 6.5(b) (b) crown is $\langle (s^1, r^1), (s^2, r^2) \rangle$ as we have $s^1 \prec r^2$ and $s^2 \prec r^1$
Fig 6.5(c): crown is $\langle (s^1, r^1), (s^3, r^3), (s^2, r^2) \rangle$ as we have $s^1 \prec r^3$ and $s^3 \prec r^2$ and $s^2 \prec r^1$

**4.观察**

- 看到消息箭头互相**“交叉”**（并且不是单纯的并发，而是有先后依赖），就要警惕 Crown。特别是两个进程互发消息，如果都是“先发后收”（发的动作都在收的动作之前），那就是最典型的 Crown。
- 非因果序的执行一定有一个皇冠
- 异步的因果序执行也会有皇冠，参考co部分的图（b）

**5.对于非RSC，Bagrodia 算法**

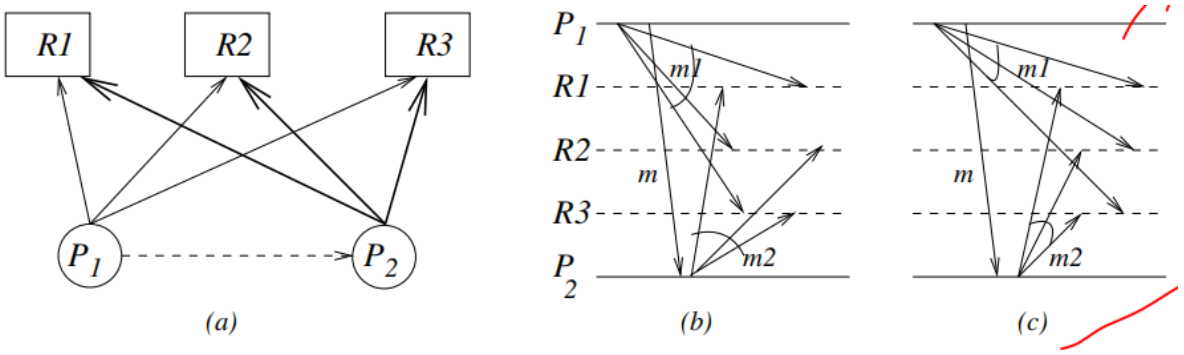
- **目标：** 在异步网络上，实现两个进程 $P_i$ 和 $P_j$ 的**同步握手**（我发给你，必须确认你收到了，我才继续）。
- **核心挑战：** 死锁。
- 
  - $P_i$ 想发给 $P_j$（阻塞等 $P_j$）。
  - $P_j$ 想发给 $P_i$（阻塞等 $P_i$）。
  - 两人都在等对方，死锁。
- **Bagrodia 算法的解决方案 (Slide 21-23)：**
  引入 **优先级 (Priority)**（通常使用进程 ID）来打破对称性。

---

# 三、 组通信与 CO 保障算法

## 1. 组通信 (Multicast) 的挑战

如果只是单播，TCP 就搞定 FIFO 了。但在组通信（Multicast）中，要保证所有接收者看到的顺序一致（Total Order）或者符合因果（Causal Order），很难，下图展示了3目标的多播可能出现的消息序。




- **全序关系：**指的是对于所有的目标对于发送进程的消息序是一致的（如图c），可以看到所有的目标上都是P2早于P1的，即使消息的发送顺序P2晚于P1（即违背CO消息序），这个系统也是满足全序关系的。
  - 简单理解就是全序关系只考虑所有目标进程消息Deliver顺序的一致性，不考虑发送进程的消息序

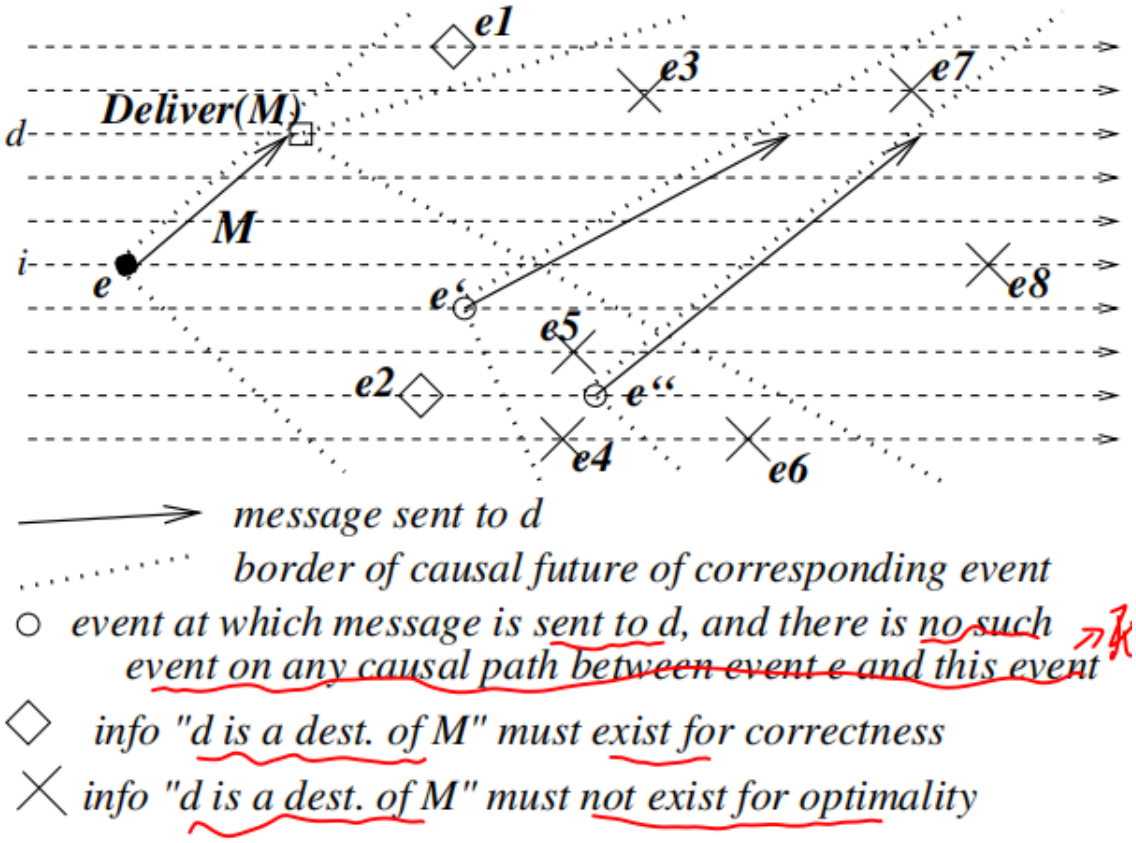
## 2.RST算法(Raynal-Schiper-Toueg Algorithm)**原理与数据结构**

- 每个进程 $P_i$ 维护两个矩阵/数组：
  1. **SENT[1..n, 1..n]**: 这是一个矩阵。`SENT[i, j]` 表示进程 $P_i$ 知道的、由 $P_i$ 发送给 $P_j$ 的消息数量。这实际上是对因果历史的压缩。
  2. **DELIV[1..n]**: 这是一个数组。`DELIV[k]` 表示本地已经投递（处理）了多少条来自 $P_k$ 的消息。
- **算法流程**
  - **发送消息时 (Send)**:
    - 发送者 $P_i$ 把自己的 `SENT` 矩阵随消息一起发出去。
    - 更新本地记录：`SENT[i, j] = SENT[i, j] + 1`。
  - **接收消息时 (Receive)**:
    - 当 $P_i$ 收到来自 $P_j$ 的消息 $M$（带着矩阵 $ST$）时，不会马上投递，而是要检查条件。

    - **安全条件 (Safety)**: `DELIV[x] >= ST[x, i]` 对所有 $x$ 成立。这句话的意思是，“消息里带的矩阵告诉我，发送者在发这条消息之前，已经知道了 $x$ 发给我的某些消息。所以我必须先把自己这边 $x$ 发给我的那些消息都处理完（`DELIV`），才能处理最新的这条消息。
    - **更新**: 一旦条件满足，更新本地的 `SENT` 矩阵（取最大值）和 `DELIV` 数组。
- **优缺点**
  - **优点**: 逻辑相对直观，不需要中心节点。
  - **缺点 (Complexity)**: 开销大！
    - 每个消息都要携带一个 $n \times n$ 的整数矩阵（$n^2$ ints/msg）。
    - 如果系统节点多（$n$ 很大），网络带宽浪费严重。

## 3. 最优 KS 算法 (Kshemkalyani-Singhal Algorithm) (Slide 26-32)

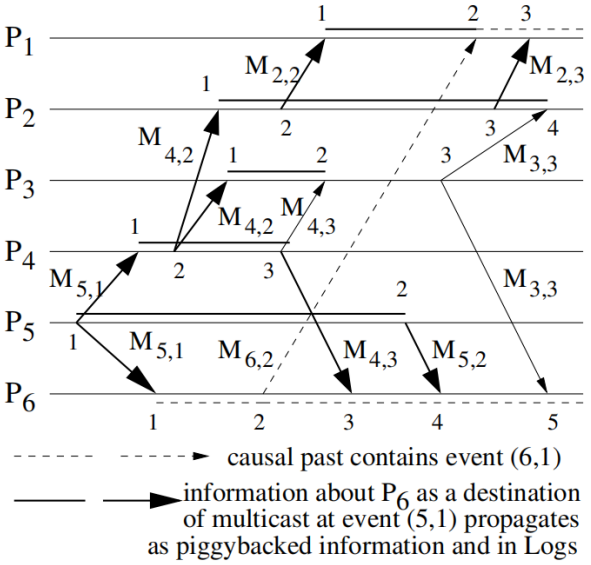
- **目标：** 实现**因果序 (Causal Order)**。
- **核心思想： 只携带必要的信息。**
  - 如果我明确知道“消息 M 已经送达目的地了”，或者“根据传递性，M 肯定会先到”，我就不需要再在后续消息里以此为据了。
  - **Log修剪 (Pruning):** 算法的核心在于如何精简 Log，不要把整个历史都塞进消息头里。
- **充要条件：**
  - 只有当“不知道 M 到了” **且** “不知道有别的消息能保证 M 的顺序” 时，才需要携带 $d \in M.Dests$ 的信息。
- **示例分析1：**




- 
  - 从上图中可以看到所有e1-e8都在消息M的future扇区
  - e3,e7,e8位于Deliver(M)扇区，所以可以确定消息已经被处理所以不需要再携带 $d \in M.Dests$

- e4,e5,e7位于e'的future扇区，而e'与消息M有**相同的接收进程**。可以通过了CO本身的消息保序特性确定e'的future扇区内的事件都发生在Deliver(M)之后，所以不必再携带 $d \in M.Dests$
- e6,e8位于e''的future扇区，原理和e'相同，所以不必再携带 $d \in M.Dests$
- 所以最终只有e1,e2需要携带 $d \in M.Dests$

- **示例分析2：**




| Message to dest. | piggybacked $M_{5,1}$.Dests |
|---|---|
| $M_{5,1}$ to $P_4$,$P_6$ | {$P_4$,$P_6$} 多播带dest |
| $M_{4,2}$ to $P_3$,$P_2$ | {$P_6$} //P4已经接收 |
| $M_{2,2}$ to $P_1$ | {$P_6$} |
| $M_{6,2}$ to $P_1$ | {$P_4$} |
| $M_{4,3}$ to $P_6$ | {$P_6$} |
| $M_{4,3}$ to $P_3$ | { } |
| $M_{5,2}$ to $P_6$ | {$P_4$,$P_6$} |
| $M_{2,3}$ to $P_1$ | {$P_6$} |
| $M_{3,3}$ to $P_2$,$P_6$ | { } |

Figure 6.13: Tracking of information about $M_{5,1}.Dests$

- 
  - 对于$M_{5,1}$，发往P4,P6，所以需要携带P4,P6
  - 对于$M_{4,2}$，已知P4已经处理，所以发往P2,P3，只需要携带P6
  - 对于$M_{2,2}$，P2本身还不确定P6是否已经送达，发往P1，继续携带P6
  - 对于$M_{6,2}$，已知P6已经处理，所以发往P1，只需要携带P4
  - 对于$M_{4,3}$，因为发往P3,P6，对于P3可以确定这个消息会在$M_{5,1}$之后处理，所以不需要携带P6（即P3已经确定了$M_{5,1}$已经完全Deliver）；但对于P6不确定$M_{5,1}$是否已经arrive，所以仍然需要携带P6
  - 对于$M_{5,2}$，P5本身不确定P4,P6是否已经处理，所以发往P6，仍然需要携带P4,P6
  - 对于$M_{2,3}$，P2本身还不确定P6是否已经处理，所以发往P1仍然需要携带P6
  - 对于$M_{3,3}$，P3进程已经确定了P4,P6已经被处理，所以发往P2,P6不需要发送任何信息，后续P2和P6发送消息时也不需要在携带$M_{5,1}$的任何信息

## 4.三阶段全序关系算法

**利用逻辑时钟（Logical Clock）进行三轮投票，确定最终的序号。**

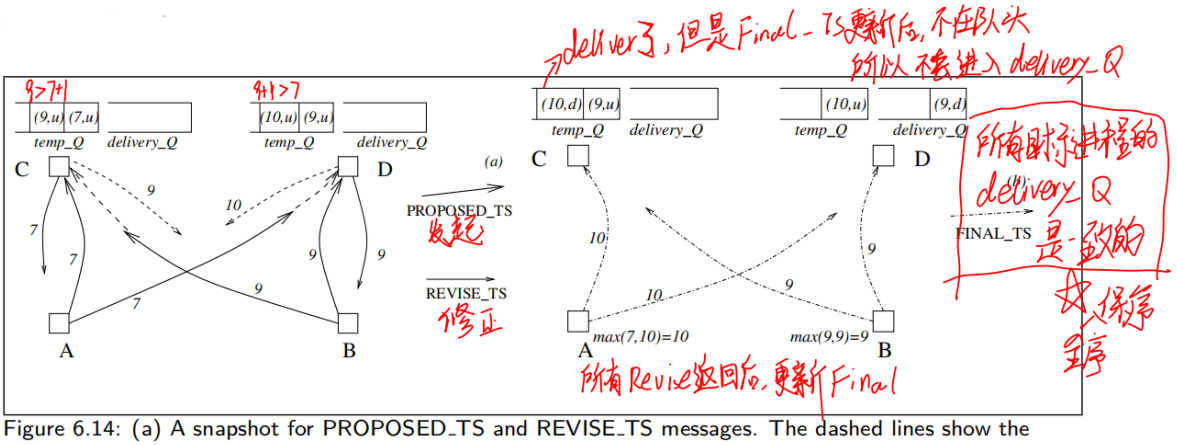



Figure 6.14: (a) A snapshot for PROPOSED_TS and REVISE_TS messages. The dashed lines show the further execution after the snapshot. (b) The FINAL_TS messages.

- **算法流程（协商过程）：**
  1. **第一阶段（发起提议，Phase 1）**：
     - 发送者 $P_i$ 给所有人发消息 $M$，并附带一个初始时间戳（REVISE_TS）。
     - 接收者收到后，**不会立刻Deliver**，而是把消息放入“临时队列（temp_Q）”，并根据自己的本地时钟给出一个**建议的时间戳**（PROPOSED_TS），发回给发送者。
  2. **第二阶段（收集决议，Phase 2）**：
     - 发送者收集所有人反馈的建议时间戳。
     - 发送者在所有建议中选一个**最大的**作为最终时间戳（Final Timestamp）。
  3. **第三阶段（最终确认，Phase 3）**：
     - 发送者把这个最终时间戳（FINAL_TS）广播给所有人。
     - 接收者收到最终时间戳后：
       1. 修改消息的时间戳为最终值。
       2. 将消息标记为“可投递（Deliverable）”。
       3. **重新排序**队列。
       4. 只有当队列**头部**的消息是“可投递”状态时，才真正交给应用程序处理。
  4. **流程简述**：
     - A向C协商7时刻（REVISED）发送，C回复7时刻（PROPOSED）可以发送；A向D协商7时刻发送，D回复最早也得在10时刻发送。
     - B向C协商9时刻发送，C回复9时刻可以发送；B向D协商9时刻发送，D回复9时刻可以发送。（以上所以消息都到达temp_Q,且不可以Deliver）
     - A确定最终的发送时刻是10(FINAL)，转达C和D；B确定最终的发送时刻是9(FINAL)，转达C和D。（这一步体现了进程确定最大的时间，来保证处于所有接受进程的Future，以此保证全序关系）
     - C的temp_Q队头是A的7时刻，转为10重新排序，tag记为可以Deliver
     - D的temp_Q队头是A的9时刻，时刻不变，直接转为Deliver，这时队头是Deliver，可以进入Deliver_Q

# 四、 多播（Multicast）

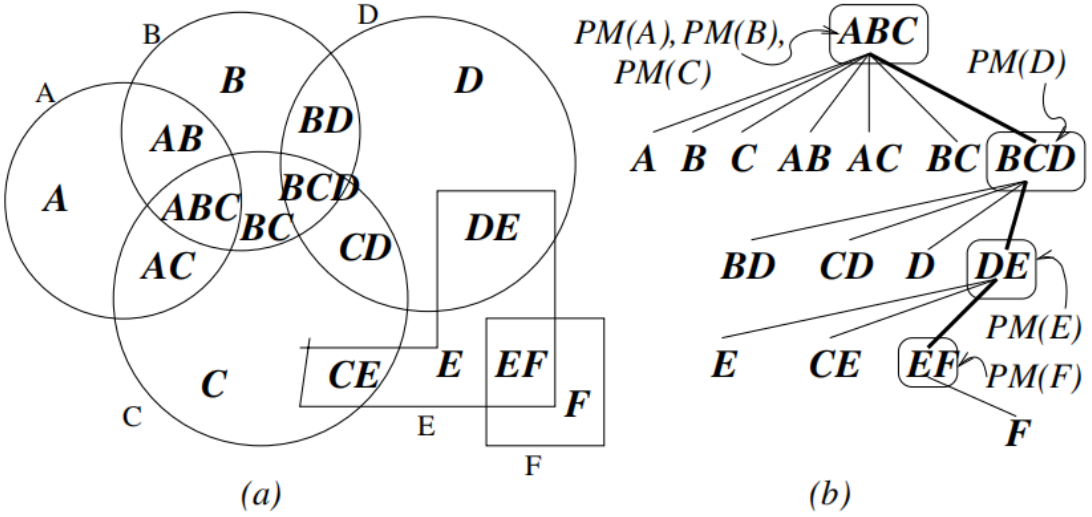
## 1.多播的分类与架构 (Slide 36)

- **SSSG / SSMG**: 单源头，发给单/多组（最简单，前面讲的算法基本都适用）。
- **MSSG**:多源单组，组内选择一个manager，采用集中式的算法即可
- **MSMG (Multiple Sources, Multiple Groups)**: 多源头，发给**多个、可能重叠**的组。
  - *难点*：如果一个人同时在 Group A 和 Group B，他收到的消息顺序该怎么算？
  - *解决方案*：引入了接下来要讲的 **Propagation Trees（传播树）**。

## 2.传播树 (Propagation Trees) (Slide 37-42)

这是一种用来解决**MSMG（多组重叠）**场景下消息排序的架构方法。

- **多播困境：**对于原始组，组与组的交集节点在通信时会违背CO
- **核心概念 (Slide 37)**：
  - **Meta-group (元组)**：把“加入组情况完全相同”的进程捆绑在一起，赋予单独的组别地位就是元组，这样就可以保证组间交集是空集，下图展示了一个复杂的组别结构的元组划分。




  对于原始组A而言，其产生的元组有{(A,B,C), (A,B), (A,C), (A)},这里的(A)指的是在原始组里直接入了A的进程。
  - **Primary meta-group (主元组)**: 在原始组内找到一个能覆盖所有元组的组作为主元组记作PM(G).图中(A,B,C)就是PM(A)
  - **树结构**: 把这些元组连成一棵树。
- **工作原理 (Slide 38-41)**：
  - 发消息时，不是直接发给所有人，而是先发给**“主元组（Primary Meta-group）”**的管理者。
  - 管理通过这棵树，一层一层往下分发消息。
  - **逻辑**：既然大家都沿着同一棵树（同一个路径）接收消息，那么顺序自然就固定了，不需要复杂的协商算法。
- **正确性 (Slide 42)**：
  - 通过树的父子关系，证明了即使在复杂的重叠组中，因果序（CO）也能得到保证。
  - 且强行把重复的组间关系，变成串行的非重复树形结构
  - “唯一的必经之路” + “关键路口的检查点（Buffer）” = 万无一失的顺序。

## 3.分类与容错 (Slide 43-45)

1. **统一可靠多播 (Uniform Reliable Multicast) 的定义**：
   - **有效性 (Validity)**：如果一个**正确**（没坏）的进程发送了消息 $M$，那么所有**正确**的进程最终都必须交付（Deliver）消息 $M$。
   - **（统一）一致性 ((Uniform) Agreement)**：如果**任何**一个进程（无论是正确的还是最终会故障的）交付了消息 $M$，那么所有**正确**的进程最终也都必须交付消息 $M$。
     - 这比普通的一致性更严格，因为它要求即使那个收到消息后马上挂掉的进程（faulty process），它如果“看到”了消息，其他人也必须能看到。
   - **（统一）完整性 ((Uniform) Integrity)**：任何进程（无论好坏）最多交付消息 $M$ 一次，且前提是 $M$ 确实被发送过。
2. **统一顺序性语义**（强调即使是故障进程也必须遵守）：
   - **（统一）FIFO 顺序**：如果一个进程先发 $M$ 后发 $M'$，任何进程（包括故障进程）除非先交付了 $M$，否则不能交付 $M'$。
   - **（统一）因果顺序 (Causal Order)**：如果 $M$ 因果上先于 $M'$，任何进程（包括故障进程）除非先交付了 $M$，否则不能交付 $M'$。
   - **（统一）全序 (Total Order)**：如果两个进程（无论好坏）都交付了 $M$ 和 $M'$，那么它们交付的先后顺序必须完全一致。
3. **时效性规范 (Timeliness Specs)**：

   基于时钟同步（全局或本地）定义的交付时间限制。
   - **实时 $\Delta$-时效性 (Real-time $\Delta$-Timeliness)**：如果消息在实时时间 $t$ 发送，任何进程交付的时间不能晚于 $t + \Delta$。
   - **本地 $\Delta$-时效性 (Local $\Delta$-Timeliness)**：如果消息在本地时间 $t_m$ 发送，任何进程交付的时间不能晚于其本地时间 $t_m + \Delta$。

## 4.多播路由算法 (Slide 46-53)

这部分从应用层下沉到了**网络层**。之前我们关心“消息顺序”，现在我们关心“如何省流量”。

- **RPF (Slide 46): 逆向路径转发。**
  - 这是一种可以在网络层防止广播风暴的简单泛洪算法。
  - 算法假设网络链路是双向对称的（即去往 $x$ 的最短路，也是 $x$ 来找我的最短路）
  - *原理*：收到包时，检查一下“这包是不是从去往源节点的最近路径发来的？”如果是，就转发；如果不是（绕路来的），就丢弃。这样就保证了即使收到了多个相同的消息，只会转发那个从最短发过来的消息，其他全部被丢弃。
- **Steiner Tree (Slide 47-48): 斯坦纳树。**
  - *问题*：我想在图中连通几个特定的点（不是所有点），怎么连总权重最小？
  - *难度*：这是一个 **NP-Complete** 问题。
  - *KMB 算法 (Slide 48)*：这就给出了一个**启发式算法**（Heuristic），虽然不是最优解，但算得快，且结果不会太差（近似比为2），越接近1越好。
- **Constrained Steiner Tree (Slide 49-52): 带延迟约束的斯坦纳树。**
  - **目标**：构建一棵树，连接源节点 $s$ 和所有目标节点 $Dest$（这些合称为 $N'$）。
  - **两个指标**：
    1. **Cost (成本 $C$)**：链路的带宽消耗或金钱成本。目标是让树的总 Cost 最小。

2. **Delay (延迟 $D$)**：链路的传输延迟。**这是一个硬性约束。**
- **约束条件**：
  从源节点 $s$ 到树上任何一个节点 $v$ 的累积延迟，必须小于给定的阈值 $\Delta$。
  $\sum_{l \in path(s,v)} D(l) < \Delta$
- *算法*：先算满足延迟的最短路，再用贪心算法把树连起来。

- **Core-based Trees (Slide 53)**: **核心树。**
  - 这是目前互联网多播常用的方法。
  - *原理*：选几个“核心节点”。想收消息的人，主动发个“Join”请求去连核心节点，路径上的树枝就长出来了。想发消息的人，把消息发给核心节点，核心节点再广播出去。

# SK快照算法详解

---

## 场景设置：红蓝双雄

- **拓扑结构**：$P_1 \leftrightarrow P_2 \leftrightarrow P_3 \leftrightarrow P_4$
- **动作**：
  - $P_1$ 发起快照，ID 为 **Red**。
  - $P_4$ 同时发起快照，ID 为 **Blue**。
  - $P_2$ 和 $P_3$ 是中间的平民。

---

## 第一阶段：各自扩张（生成树的形成）

1. $P_1$ **(红王) 行动**：
   - $P_1$ 记录自己的 $LS_1$。
   - $P_1$ 变成 **红色** (`master = P1`)。
   - $P_1$ 向 $P_2$ 发送 **Red Marker**。
2. $P_4$ **(蓝王) 行动**：
   - $P_4$ 记录自己的 $LS_4$。
   - $P_4$ 变成 **蓝色** (`master = P4`)。
   - $P_4$ 向 $P_3$ 发送 **Blue Marker**。
3. $P_2$ **接收 Red Marker**：
   - $P_2$ 还是白纸一张。收到红色标记，它决定效忠 **红色**。
   - $P_2$ 记录 $LS_2$，并把 $P_1$ 认作父节点。
   - $P_2$ **属于红色生成树。**
   - $P_2$ 向邻居 $P_3$ 转发 **Red Marker**。
4. $P_3$ **接收 Blue Marker**：
   - $P_3$ 也是白纸一张。收到蓝色标记，它决定效忠 **蓝色**。
   - $P_3$ 记录 $LS_3$，把 $P_4$ 认作父节点。
   - $P_3$ **属于蓝色生成树。**
   - $P_3$ 向邻居 $P_2$ 转发 **Blue Marker**。

---

## 第二阶段：边界冲突（回答你的核心疑问）

现在到了最关键的时刻：$P_2$ **和** $P_3$ **在中间相遇了。**

### 场景焦点：$P_2$ (红色) 收到了 $P_3$ 发来的 Blue Marker

此时 $P_2$ 的状态是：我已经拍过照了，我是红色的。

**问题 1：收到 Blue Marker 会封闭当前信道吗？**
**答案：会！绝对会！**

- **CL 规则依然适用：** Marker 的物理意义是"分界线"。无论这个 Marker 是红是蓝，是圆是扁，只要收到 Marker，就意味着 **"通道 $C_{3\to2}$ 的录制结束"。**
- **动作：** $P_2$ 停止录制从 $P_3$ 发来的消息。此前录制的内容（如果有）就是 $CS_{3\to2}$。

**问题 2：$P_2$ 会变成蓝色吗？**
**答案：不会。**

- $P_2$ 发现 Marker 里的 ID 是 **Blue**，而自己的 ID 是 **Red**。
- **动作：** $P_2$ 拿出小本本（`id-border-set`），记下一笔：**"我发现我的邻居 $P_3$ 属于 Blue 阵营。"**

**问题 3：这个时候的 LS 和 CS 是什么样的？**
假设在 $P_3$ 拍完照后，但在发 Blue Marker 之前，$P_3$ 偷偷给 $P_2$ 发了一个消息 $m_{data}$。

- **对于 $P_2$ 来说：**
  - **LS (Local State):** 是它当初收到红旗时拍的照片。
  - **CS (Channel State of $C_{3\to2}$):** 包含了消息 $m_{data}$。因为这个消息是在 $P_2$ 变红之后、收到 Blue Marker 之前到达的。

---

## 第三阶段：为什么有两个生成树？怎么合并？

### 为什么有两个树？

因为系统里有两个**并发的发起者**。

- **树 1 (红色)：** Root=$P_1$, Child=$P_2$。
- **树 2 (蓝色)：** Root=$P_4$, Child=$P_3$。
  这就好比两个墨水滴在纸上，各自扩散，直到边缘相撞。

### 怎么合并？

1. **上报：**
   - $P_2$ 告诉 $P_1$："老大，我的任务完成了，还有个情报：**我接触到了 Blue 阵营。**"
   - $P_3$ 告诉 $P_4$："老大，任务完成，情报：**我接触到了 Red 阵营。**"
2. **握手：**
   - $P_1$ (红王) 收到情报，知道世界上还有一个 $P_4$ (蓝王) 也在拍照。
   - $P_1$ 主动联系 $P_4$（或者 $P_4$ 联系 $P_1$）。
3. **交换：**
   - $P_1$ 把手里持有的 **红色半壁江山 ($LS_1, LS_2, CS_{...}$)** 打包发给 $P_4$。
   - $P_4$ 把手里持有的 **蓝色半壁江山 ($LS_3, LS_4, CS_{...}$)** 打包发给 $P_1$。

4. **结局：**
   - $P_1$ 和 $P_4$ 手里都拥有了 $\{LS_1, LS_2, LS_3, LS_4, \dots\}$。
   - 这就构成了一个**全局一致**的快照。

---

## 总结图解

```
[P1 红] --> [P2 红]  <--边界-->  [P3 蓝] <-- [P4 蓝]
Initiator    Child                Child      Initiator
```

1. $P_2$ **收到蓝 Marker：**
   - **信道：** 封闭 $C_{3\to2}$（停止录像）。
   - **状态：** 保持红色不变。
   - **记录：** 记下 `Border = Blue`。
2. $P_3$ **收到红 Marker：**
   - **信道：** 封闭 $C_{2\to3}$（停止录像）。
   - **状态：** 保持蓝色不变。
   - **记录：** 记下 `Border = Red`。
3. **合并：** $P_1$ 和 $P_4$ 也就是这两个树的**根**，负责最后的数据交换。

# 汇聚树（Sink Tree）与分布式Floyd-Warshall算法

**Sink Tree of Pivot:** 网络中所有"能走到 $pivot$"的最短路径集合，构成了一棵以 $pivot$ 为根（终点）的树。所有水流（数据）最终都流向 $pivot$。 既然大家平时都能顺着这棵树走到 $pivot$，那 $pivot$ 就可以**逆着这棵树**，把它的数据（$LENGTH[pivot, *]$）分发给所有人。

**Sink Tree 的构建与确认（握手过程）：**
这对应 PPT P40 的中间部分和 P43 的代码。
在第 $pivot$ 轮迭代开始时，节点之间会进行一次"认亲"：

1. **我属于谁的树?**
   - 每个节点 $i$ 查表：我去往 $pivot$ 的下一跳（Next Hop / Parent）是谁？假设是 $B$。
   - 那我就给 $B$ 发一个消息：`IN_TREE`。意思是："大佬，$B$，我是你的下线，你要是收到了 $pivot$ 的数据，记得转发给我。"
   - 如果你不是我去 $pivot$ 的下一跳，我就发 `NOT_IN_TREE`。
2. **确定孩子 (Children)：**
   - 节点 $B$ 收到 $i$ 的 `IN_TREE`，就把 $i$ 记为自己的"孩子"。
   - 节点 $B$ 收到 $k$ 的 `NOT_IN_TREE`，就知道不用管 $k$。

这个过程就是一种**同步 (Synchronization)**，确保路铺好了，数据才能发。

## 分布式 Floyd-Warshall 完整流程 (PPT P43 代码解析)

**最外层循环：** `for pivot = 1 to n` （全网同步进行，假设有机制保证轮次）

## 第一阶段：建立/确认 Sink Tree (Line 2-6)

- 我是节点 $i$，我知道要去 $pivot$ 下一步该给谁（查 `PARENT[pivot]`）。
- 我给那个“父节点”发 `IN_TREE`。
- 同时，我等待我的所有邻居给我发消息。如果邻居发给我 `IN_TREE`，我就把它加到我的转发列表里。

## 第二阶段：Pivot 广播数据 (Line 7-14)

- **如果你是 pivot 本人：** 你是源头。把你的路由表 $LENGTH[1..n]$ 也就是代码里的 `PIVOT_ROW` 打包成消息 `PIV_LEN`，发给所有刚才认你做“爸爸”的邻居。
- **如果你是普通节点：**
  - 你从你的“父节点”那里收到 `PIV_LEN`（这是 pivot 的数据）。
  - 你把这份数据**复制转发**给你的所有“孩子”（刚才给你发 IN_TREE 的那些邻居）。

## 第三阶段：本地计算 (Line 15-18)

- 现在，节点 $i$ 终于拿到了梦寐以求的 `PIVOT_ROW`（也就是 $LENGTH[pivot, *]$）。
- 开始跑那个经典的循环：
  `for t = 1 to n`
  `if LEN[pivot] + PIVOT_ROW[t] < LEN[t]`
  更新我的 `LEN[t]`。

# 首领选举HS算法示例

假设环是： **5 — 8 — 3 — 9 — 2** (顺时针)

**Round 0 (距离 $2^0 = 1$):**

- **5号：** 向左探2，向右探8。
  - 遇到8（8>5），5的消息被丢弃。5淘汰。
- **8号：** 向左探5，向右探3。
  - 5<8，3<8。8收到两边的回复。**8晋级 Round 1。**
- **3号：** 遇到8和9，两头都被丢弃。3淘汰。
- **9号：** 向左探3，向右探2。
  - 3<9，2<9。9收到两边的回复。**9晋级 Round 1。**
- **2号：** 遇到9，被丢弃。2淘汰。

**结果：** 第一轮刷掉了 5, 3, 2。剩下 **8** 和 **9** 两个大佬进入下一轮。

**Round 1 (距离 $2^1 = 2$):**

- **8号：** 探测范围是2跳。
  - 往右走：8 -> 3(过) -> **9**。遇到9，9>8。8的消息被丢弃。
  - 8号没收到回复，淘汰。
- **9号：** 探测范围是2跳。
  - 往右走：9 -> 2(过) -> 5(过)。5回话给9。
  - 往左走：9 -> 3(过) -> 8(过，因为此时8是作为接收者，8<9)。8回话给9。

- 9号收到两边的回复，**9晋级 Round 2**。

**Round 2 (距离 $2^2 = 4$):**

- **9号**： 探测范围是4跳。因为只有5个点，消息会绕一圈回到9。
- 9号收到自己的ID，宣布当选 Leader。

# Bagrodia算法实现

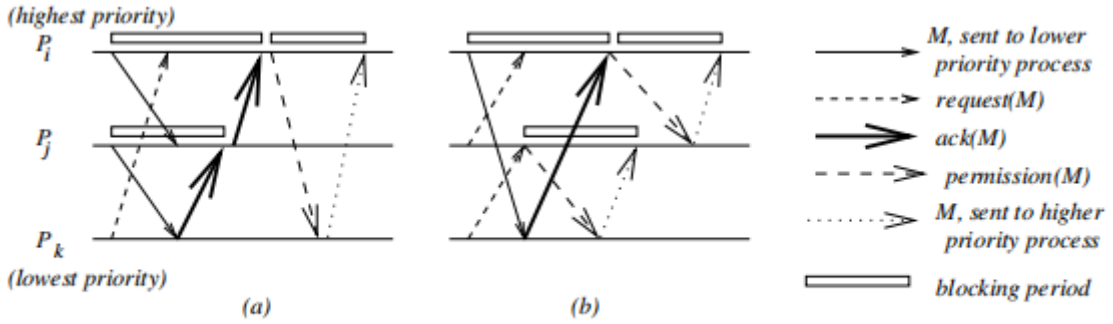



Figure 6.10: Scheduling messages with sync communication.

假设 $P_i$ **(高优先级)** 和 $P_j$ **(低优先级)** 同时想给对方发消息。

1. $P_i$ **(高) 的动作：**
   - 向 $P_j$ 发送消息M。
   - 然后 $P_i$ 进入**阻塞状态**，等待 $P_j$ 的发回确认消息。
   - $P_i$ 收到$P_j$的回复ACK(M),结束阻塞状态
   - $P_i$ 回复 `Permission` 给 $P_k$，再次进入阻塞状态
   - $P_i$ 收到$P_k$发来的消息M'',结束阻塞状态
2. $P_j$ **(中) 的动作：**
   - 向 $P_k$ 发送消息M'。
   - $P_j$ 进入阻塞，收到了 $P_i$ 的消息M。
   - $P_j$ 收到$P_k$的回复ACK(M')。
   - $P_j$ 回复 $P_i$ 的消息M，发送ACK(M)。
3. $P_k$ **(低) 的动作：**
   - 向 $P_i$ 发送 `Request(M)`。
   - $P_k$ 回复 $P_j$ 的消息M'，发送ACK(M')。
   - **$P_k$ 收到$P_i$回复 `Permission`,向$P_i$发送消息M''

**总结：** 通过让“低优先级”的人让路，避免了两人互相死等的死锁。

# KMB 算法的执行过程

我们可以把它想象成在一个地图上修路，连接几个指定的关键节点（Steiner points），虽然地图上有很多其他非关键城市。

- **Step 1: 构建闭包图 / 距离图 (Construct Closure Graph $G'$ )**
  - **做什么**：算出所有**关键节点**（Steiner points, $N'$）两两之间的**最短路径**距离。

- **结果**：构建一个新的“虚拟图” $G'$。这个图里只有关键节点，每两个关键节点之间都有一条边，边的权重就是它们在原图中的最短路径（shortest path）大小。
- *通俗理解*：暂时忽略中间经过的小村庄，只看关键城市之间直达要走多远。

- **Step 2: 在虚拟图上找最小生成树 (MST on $G'$)**
  - **做什么**：在刚才建立的虚拟图 $G'$ 上运行 MST 算法（如 Prim 或 Kruskal）。
  - **结果**：得到一棵树 $T'$，它告诉我们**哪些关键城市应该直接连接**，**总距离在虚拟层面是最小的。**
- **Step 3: 还原回原图 (Construct Subgraph $G_s$)**
  - **做什么**：把 $T'$ 中的每一条虚拟边，替换回原图 $G$ 中对应的真实最短路径。
  - **结果**：得到原图的一个子图 $G_s$。
  - *注意*：这时候*可能会出现环路*，因为不同的最短路径可能会共用某些物理路段。
- **Step 4: 再次找最小生成树 (MST on $G_s$)**
  - **做什么**：在子图 $G_s$ 上再跑一次 MST 算法。
  - **结果**：得到 $T_s$。
  - *目的*：这一步是为了消除 Step 3 中可能产生的环路，并处理路径重叠的情况，进一步降低成本。
- **Step 5: 剪枝 (Pruning)**
  - **做什么**：检查 $T_s$ 中的叶子节点（树梢）。如果某个叶子节点**不是**关键节点（Steiner point），就把它剪掉。重复这个过程直到所有叶子节点都是关键节点。
  - **结果**：最终的启发式斯坦纳树 $T_{Steiner}$。
  - *通俗理解*：修剪掉那些伸出去的死胡同，如果死胡同尽头不是我们要去的关键城市，那段路就没必要修。

# 带最大时延限制的Steiner Tree算法

## 第一步：计算闭包图 (Compute Closure Graph $G'$)

- **目的**：把原图简化。算出所有**关键节点**（Steiner Points）之间在满足延迟约束下的**最便宜路径**。
- **概念**：**受限最廉价路径 (Constrained Cheapest Path)**。
  - 通常的最短路算法（如 Dijkstra）只看一个权重。
  - 这里我们需要找：在延迟不超过 $\Delta$ 的所有路径中，成本最低的那一条。
- **怎么算 (动态规划)**：
  - PPT 提到了类似 Floyd 算法的 DP 思路。
  - 定义 $C_d(x, y)$：从 $x$ 到 $y$ 且延迟**刚好为** $d$ 的路径的最小成本。
  - 最终，两点间的受限最廉价路径 $P_C(x, y) = \min_{d<\Delta} C_d(x, y)$。
- **结果**：得到一个完全图 $G'$，任意两个关键节点之间都有一条边，边的权重就是它们之间的“受限最廉价路径成本”。

## 第二步：贪心构建生成树 (Construct Constrained Spanning Tree)

- **目的**：在 $G'$ 上选边，构建一棵树 $T$。
- **过程**：
  1. 初始化树 $T$ 只包含源节点 $s$。
  2. 不断往树里加节点。假设 $u$ 已经在树里，我们要选一个还不在树里的节点 $v$ 加入。
  3. **关键问题**：选哪个 $v$，连哪条边 $(u, v)$？
  4. 这需要用到**选择策略（启发式公式）**，见下文详述。

## 第三步：展开与去环 (Expand and Prune)

- 把 $T$ 中的虚拟边还原成原图里的真实路径。
- 如果还原过程中形成了环路（Loop），就把环断开，删除多余的边。

---

## 核心：怎么选边？(两类 Heuristics)

这是 PPT 第50页最难理解的部分。在第二步中，如何决定把哪个点加入树？算法提供了两个"打分公式"来权衡 Cost 和 Delay。

假设：

- $u$ 是树上的点。
- $v$ 是树外的点。
- $P_D(s, u)$ 是源点到 $u$ 目前已知的延迟。
- $\mathcal{D}(u, v)$ 是边 $(u, v)$ 的延迟。

### 策略一：$CST_{CD}$ (Cost-Delay 权衡)

公式：
$f_{CD}(u, v) = \frac{C(u,v)}{\Delta - (P_D(s,u) + \mathcal{D}(u,v))}$

- **分子 ($C(u, v)$)**：增加的成本。我们希望它越小越好。
- **分母 ($\Delta - \ldots$)**：**剩余的延迟预算 (Residual Delay)**。也就是"我还剩多少时间可以挥霍"。我们希望剩余时间越多越好（即分母越大越好），这样后续加点更容易。
- **逻辑**：我们要找 $f$ 值**最小**的边。
  - 这意味着：**成本要低，且剩下的时间预算要多。**
  - 如果某条边加进去后，延迟马上就要爆表了（分母接近0），分数值会变得无穷大，算法就不会选它。

### 策略二：$CST_C$ (只看 Cost)

公式：
$f_C = C(u, v) \quad (\text{前提是延迟不超标})$

- **逻辑**：只要加上这条边后总延迟不超过 $\Delta$，我就只选最便宜的那条。
- **特点**：比较简单粗暴，容易陷入局部最优（为了省钱选了延迟较高的路，导致后面剩下的时间预算不足，没法连接远处的节点）。

---

## 算法复杂度 (Complexity)

- **Step 1 (计算路径)**：$O(n^3\Delta)$
  - 这是最耗时的。因为它是基于整数延迟值的动态规划。如果 $\Delta$（允许的延迟上限）很大，这个步骤会非常慢。
- **Step 2 (建树)**：$O(k^3)$，其中 $k$ 是关键节点数量。
- **Step 3 (还原)**：$O(kn)$。

**结论**：整个算法的瓶颈在于**第一步**。虽然慢，但它能有效地在“省钱”和“不卡”之间找到平衡。

## 示例

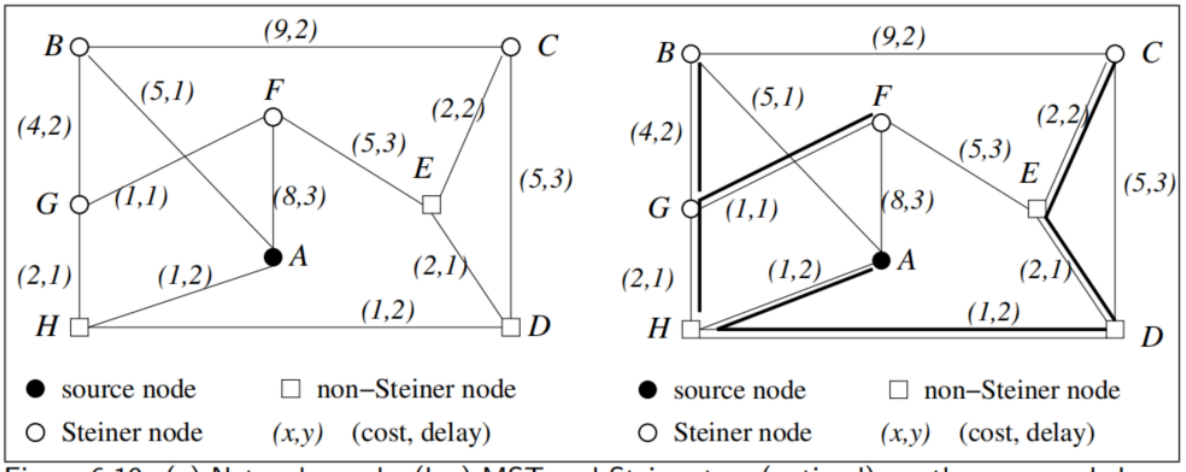



Figure 6.19: (a) Network graph. (b,c) MST and Steiner tree (optimal) are the same and shown in thick lines.

- 第一步是找到所有关键节点之间不超过时延的最短路径
  - A: B 5 C 6 F 4 G 3
  - B: C 9 F 5 G 4
  - C: F 7 G 7
  - F: G 1
- 第二步是构建所有可达路径不超过时延的MST（选择的是后，要考虑启发因子）
  - 从A开始，先选择G A→H→G
  - 根据G选新节点，选择F，G→F
  - 根据F选新节点，选择B，F→G→B
  - 根据B只能连接C，B→C
- 第三步是展开与断环
  - 展开的我们已经在第二步中完成了，原始的第二步只会根据启发因子计算应该连接的点，而不会展开真实路径
  - 由于真实的路径存在一个环路，所以断开其中cost最大的边B→C
  - 此时B, C仍然可达，只是路径变成了B→G→H→D→E→C。

# Core-based Trees

- **之前的算法（如 Steiner Tree）**：是**Source-based（基于源）**的。也就是说，如果群里有10个人要发言，网络就要为这10个人分别建立10棵不同的树。这非常浪费路由器的内存和计算资源。

- **CBT算法**：是**Shared Tree（共享树）**的。不管谁发言，大家都共用这一棵树。这棵树有一个中心点，叫**Core（核心节点）**。

---

## 1. 树是如何建立的？（Building the Tree)

CBT 的树不是一开始就铺好的，而是有人想听（接收者）的时候才长出来的。

- **步骤 ①：发起加入 (Join)**
  - 假设你是一个接收者（Receiver），你想加入某个组。你不需要知道谁是发送者，你只需要知道这个组的 **Core（核心节点）** 是谁。
  - 于是，你向 Core 发送一个单播的 `join` 消息："嘿，我要入群"。
- **步骤 ②：嫁接 (Grafting) —— 核心机制**
  - 这个 `join` 消息会沿着网络向 Core 跑去。
  - **关键点**：它不一定非要跑到 Core 那里！
  - **拦截机制**：在去往 Core 的路上，如果遇到某台路由器（节点）**已经在树上了**（说明之前有别人的 `join` 消息经过这里），那么这个 `join` 消息就会停下来。
  - **嫁接**：你的路径就会直接"接"在这个节点上，成为树的一个新分支。
  - *效果*：这就像树根寻找水源一样，只要碰到了主根系，就融为一体。这样避免了重复建设路径。

---

## 2. 消息是如何传输的？（Data Transmission)

树建好了，怎么发消息呢？分两种情况：

- **步骤 ③：树上的人发消息 (On-tree Sending)**
  - 如果发送者本身就在这棵树上（比如它也是接收者）。
  - 它直接把消息发给它在树上的邻居（父节点和子节点）。消息就像水流一样，沿着树枝流遍整棵树。这叫 **Flooding on the core tree**。
- **步骤 ④：树外的人发消息 (Off-tree Sending)**
  - 如果发送者不在树上（它只想说话，不想听，或者刚开始说话还没入树）。
  - 它并不知道树长什么样，但它知道 **Core** 在哪。
  - **动作**：它把消息封装成单播包，发向 Core。
  - **触发**：这个包在去往 Core 的路上，一旦**撞到了**树上的任何一个节点（不一定是 Core 本身），这个节点就会把单播包解开，变成多播消息，然后按照**步骤 ③** 的方式在树上泛洪。

---

## 3. CBT 的优缺点总结（PPT外延知识)

- **优点**：
  - **省资源**：全网只需要维护一棵树，路由表项很少（$O(1)$，而不是 $O(N)$）。
  - **可扩展性强**：非常适合那种成员很多、但在大范围稀疏分布的组。
- **缺点**：
  - **流量集中**：Core 节点容易成为瓶颈（所有人都往它那里发）。

- **路径非最优**：有时候两个邻居通信，消息非要绕到 Core 再回来（所谓的“狗腿路由” Dog-leg routing），延迟可能比 Steiner Tree 高。
- **单点故障**：如果 Core 挂了，整棵树可能需要重构（虽然有 Backup Core 机制）。

**一句话总结 Slide 53**：
CBT 就是在网络中心选个“盟主”（Core），谁想听就修路连向盟主（半路遇到盟主的人也算连上），谁想说就朝着盟主喊话，利用这棵共享的树来分发消息，从而节省网络资源。